

Noah Amack

Carlos D Alvarado-Lopez

Reef Starnes

Solar-Glide VTOL Drone

Carlos Alvarado-Lopez
Reef Starnes
Noah Amack

ECE Senior Design Capstone Project Document
School of EECS
Oregon State University
19 October 2025

Table of Contents

Table of Contents.....	2
Executive Summary.....	4
Diversity, Equity, and Inclusion (DEI) Statement.....	6
1. Project Overview.....	7
1.1. Description.....	7
1.2. Gap Analysis.....	8
1.3. Design Impact Assessment.....	9
1.4. References.....	10
2. Top-Level Project Specification.....	12
2.1. Block Diagrams.....	12
2.2. Block Descriptions.....	13
2.3. Interface Definitions.....	16
2.4. References.....	21
3. Block Designs.....	22
3.1. Prop Control.....	22
3.1.1. Description.....	22
3.1.2. Design.....	23
3.1.3. General Validation.....	24
3.1.4. Interface Validation.....	25
3.1.5. Verification Process (For one ESC and one motor).....	29
3.1.6. References.....	31
3.2. Communication.....	32
3.2.1. Description.....	32
3.2.2. Design.....	33
3.2.3. General validation.....	34
3.2.4. Interface Validation.....	35
3.2.5. Verification Process.....	36
3.2.6. References.....	38
3.3. Code block.....	39
3.3.1. Description.....	39
3.3.2. Design.....	40
3.3.3. General Validation.....	42
3.3.4. Interface Validation.....	43
3.3.5. Verification Process.....	45
3.3.6. References.....	46
3.4. Power Distribution Block.....	47

3.4.1. Description.....	47
3.4.2. Design.....	48
3.4.3. General Validation.....	50
3.4.4. Interface Validation.....	50
3.4.5. Verification Process.....	55
3.4.6. References.....	58
3.5. Flight Control.....	60
3.5.1. Description.....	60
3.5.2. Design.....	60
3.5.3. General Validation.....	68
3.5.4. Interface Validation.....	69
3.5.5. Verification Process.....	73
3.5.6. References.....	77
3.6. Aircraft-Body.....	79
3.6.1. Description.....	79
3.6.2. Design.....	80
3.6.3. General Validation.....	84
3.6.4. Interface Validation.....	85
3.6.5. Verification Process.....	86
3.6.6. References.....	86
4. Project Closing.....	88
4.1. Future Recommendations.....	88
4.1.1. Technical recommendations.....	88
4.1.2. Global impact recommendations.....	88
4.1.3. Teamwork recommendations.....	88
4.2. Project Artifact Summary with Links.....	88
4.3. Presentation Materials.....	88
4.4. References.....	89
A. Appendix.....	89

Executive Summary

Our Solar-Glide VTOL Drone project aims to develop a long-endurance, cost-effective unmanned aerial vehicle (UAV) capable of vertical take off and landing (VTOL) with the added capability of gliding with its fixed wing aerodynamic design. The team is currently developing the drone through a series of integrated stages that involve research and design. These stages include flight control design, power system development, and the communication setup for ground monitoring. The flight control system is managed by an onboard microcontroller PCB that processes sensor data from the barometer (altitude) and power efficiency data collected from the solar charging rate. The communication system will use telemetry to send this data to a remote controller on the ground for live monitoring. Power management is handled with a lithium polymer battery system supported by the solar panels. The project is in the design and research phase, with prototyping planned for the near future.

Regarding current achievements, there are a few to list in these preliminary stages. For one, the team successfully laid out the block-level design, establishing a basis for how the different systems interact with the power or control signal interfaces of each block. For two blocks, "Power Supply", "Flight Controller", and "Prop Control" have been assigned to team members Carlos, Reef, and Noah, respectively. We have achieved basic component selection, such as ESP32 MCU, with plenty of compatible code frameworks to utilize, e.g., "esp-drone" [1] and "FREERTOS" [2]. But the greatest achievement of all is our established lines of communication and regular meeting times with the team.

The ultimate challenge is flight. The first term will establish the electronics framework for experimenting with aerodynamically capable body designs in the second term. The preliminary stages of this project are challenged by precise component selection. For instance, the power supply block requires, at a minimum, 65% power efficiency excluding the solar panels (where efficiencies hover about 15% [3]). This is a challenge in itself, as our team in the past has employed poor efficiency linear regulators, and now must design a greater efficiency means of conversion, not entirely familiar to us, a buck converter. Another challenge is designing a custom MCU board for flight control. Although every member of the team is greatly familiar with the ins and outs of development boards, this is a goal above that. Overall, the system and its associated electronics must be reasonably compact and lightweight to achieve gliding status.

The next steps require some component selection, the specific DC motors we will use, especially so for understanding the nominal voltage and current draw to design the power supply for, and the specifics of the dsig interface with this block and MCU. Aside from this, we expect to begin PCB design starting week 4.

Noah Amack

Carlos D Alvarado-Lopez

Reef Starnes

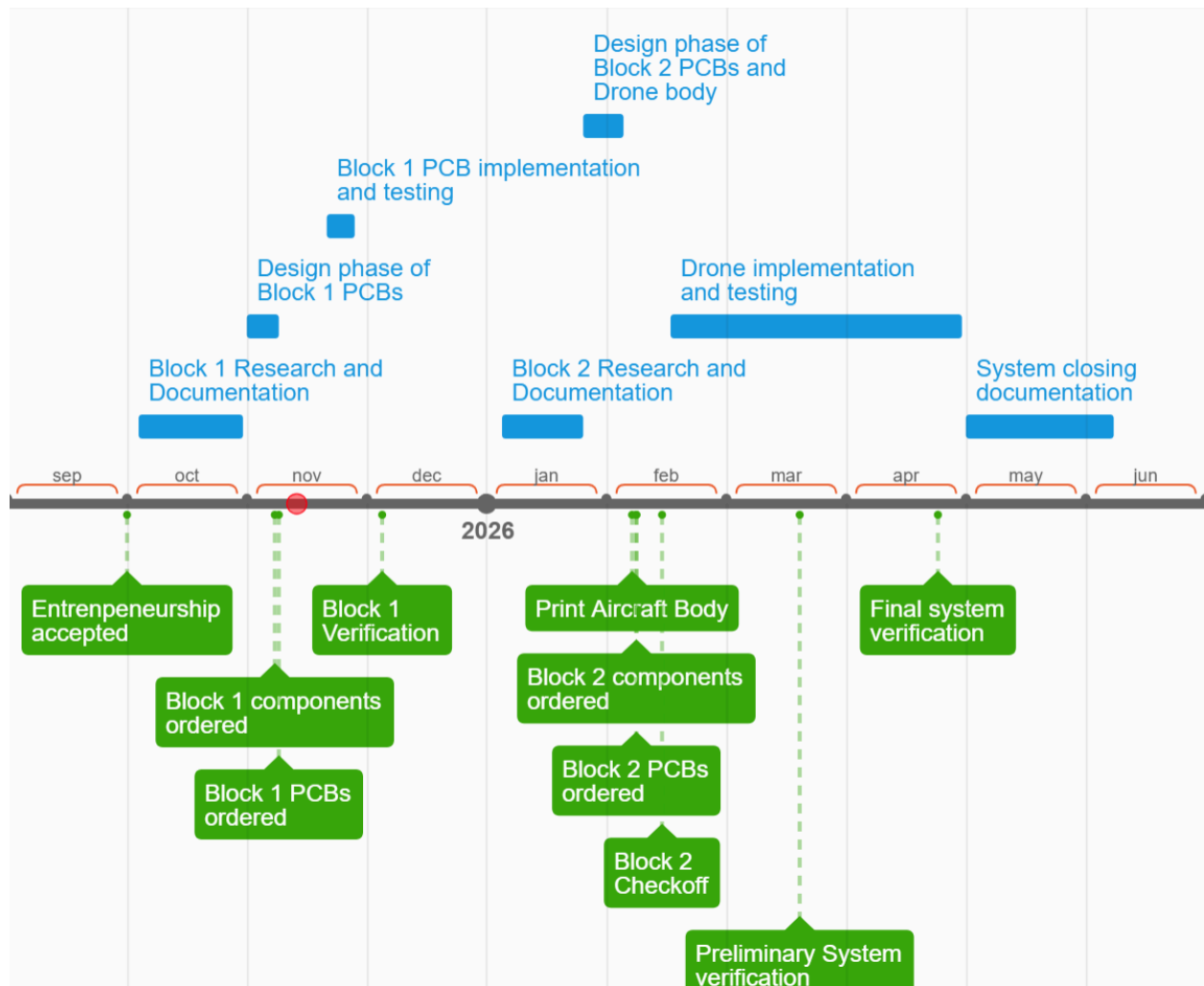


Figure 1: Project Timeline.

Diversity, Equity, and Inclusion (DEI) Statement

Our project is committed to valuing all voices and perspectives throughout the design and development process. Along with striving to treat all people equally and approaching every decision with fairness and respect, we recognize that choices such as battery selection, PCB fabrication, and testing practices have environmental and safety implications that can affect different groups unevenly. Because of this, we aim to design technology that benefits society broadly and minimizes unnecessary harm. We will strive to follow our set teamwork approaches of communication, work ethic, and respect. Above all, we emphasize treating every individual as an equal contributor in the pursuit of innovation, guided by merit.

1. Project Overview

The Project Overview section introduces the Solar-Glide VTOL Drone, its primary objectives, and the technical requirements implicit in its design. The Solar-Glide is a long-endurance, solar-assisted unmanned aerial vehicle capable of both fixed-wing gliding and vertical takeoff. Its purpose is to provide a cost-effective, energy-efficient platform for research, surveying, and exploration in settings that require the best of both worlds. This section additionally outlines the systems engineering requirements, analyzes why our project is needed, and evaluates its impacts.

Engineering Requirements:

1. The system must glide at least 4 meters.
2. The system shall recharge its onboard batteries from a nominal 120 V AC, 60 Hz single-phase supply compatible with standard North American receptacles (NEMA 1-15/5-15).
3. The system must be controlled wirelessly from at least 50 meters.
4. The system must include independent charging.
5. The system must remain airborne for at least 15 minutes on a single charge.
6. The system shall vertically ascend to an altitude of at least 4 meters above ground level.
7. The system must relay the battery charge back to ground.
8. The system must be lightweight with a net weight of less than 6000 grams.

1.1. Description

Our Solar-Glide VTOL Drone project aims to develop a long-endurance, cost-effective unmanned aerial vehicle (UAV) capable of vertical take off and landing (VTOL) with the added capability of gliding with its fixed wing aerodynamic design. The purpose of this document is to present the design's objectives, development process, and the final working system.

The purpose of this project is to develop an omnidirectional, long-endurance, cost-effective unmanned aerial vehicle (UAV) as a proof-of-concept platform for researchers, surveyors, and explorers alike to collect data. It can take off and land vertically with its quadcopter design and transition into forward gliding with its propellers off, significantly increasing the drone's energy efficiency and endurance. Furthermore, integrated solar panels will recharge the onboard battery mid-flight, extending its operational range beyond the typical limits of conventional drones on the market. This project displays a cost-effective, innovative approach to enhancing a drone's endurance with hybrid power integration and an aerodynamically efficient design, resulting in a versatile platform for research, exploration, surveillance, or environmental applications.

The project development process followed an iterative engineering design approach. Initially, our team used online research to gauge the feasibility of this project better. The critical components, on which the whole design leans, were then selected. These include but are not limited to, motors, solar panels, battery, sensors, ground-aircraft communication methods, and MCU. These components are most crucial, as we must design for compatible voltages, current draw/supply, and weight. The fall term consists of developing the power supply, flight controller, and prop control blocks and their associated PCBs. More on the timeline can be found in the Executive Summary section above.

1.2. Gap Analysis

Conventional fixed-wing drones offer endurance but lack hover and maneuverability, while quadcopters provide control but suffer from limited battery life. The intended end users for our project include researchers and surveyors who require a versatile, efficient, and cheap platform for aerial observation. The Solar-Glide VTOL Drone project exists to address a significant practical and technological gap in the UAVs of today. This is due to the lack of affordable, long-endurance drones that are capable of maneuvering in obstacle-stricken environments. Conventional drones used in research, exploration, and data collection are often constrained by these constraints. According to a 2023 Grand View Research study, the global drone market is expected to triple in value in 2030 [4]. This can be seen in Figure 2 below, and demonstrates that drones and their variations are in demand.

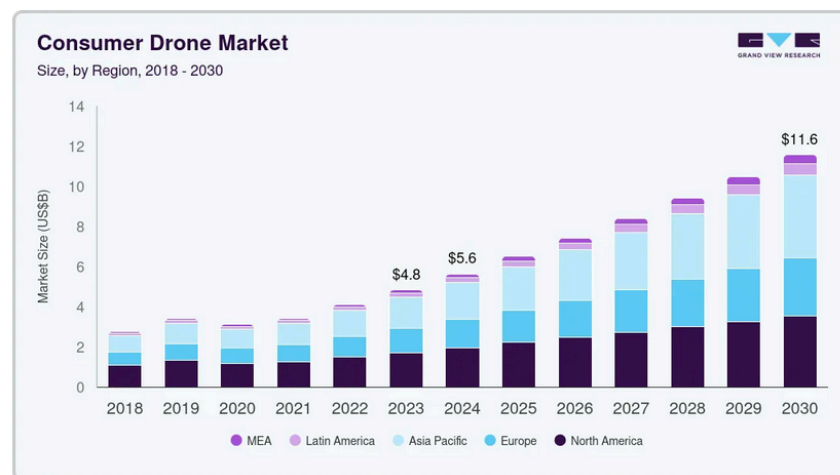


Figure 2: The global consumer drone market size shows a rapid upward growth trend according to a 2023 study by Grand View Research [4]

Furthermore, the average consumer-grade drone has a maximum flight time of roughly 30 minutes [12]. This limitation can be overcome with our design, which does not constantly fight the downforce of gravity, but instead floats on the air itself.

Beyond the clear market backing, there are logical constraints with typical drones that this project addresses. For example, NASA's Earth Observatory stated that, "The skies above the Amazon are almost always churning with clouds and storms, making the basin one of the most difficult places for scientists to map and monitor." [5]. The Amazon's vastness, thick trees that form a canopy, demand a versatile hybrid option for collecting data. Although this is one example, it demonstrates the complexity of different environments that demand a solution.

Our intended market consists primarily of drone manufacturers and UAS companies seeking to improve endurance and energy efficiency in their aircraft. These companies are actively looking for innovative solutions to improve their products, such as solar-assisted charging and gliding capabilities. Our solar-glide VTOL design demonstrates a functional system that integrates solar power, onboard charging, and safe power distribution into a modular architecture that manufacturers can adopt or further develop. As mentioned before, further markets can include research and product use, with specific cases such as agricultural monitoring, environmental surveying, and infrastructure inspection. Potential shareholders include drone companies, industrial technology investors, and research groups interested in next-generation drone systems.

1.3.Design Impact Assessment

The purpose of this statement is to evaluate the potential impacts of the Solar-Glide VTOL Drone project from environmental, safety, and economic perspectives. By analyzing these aspects, the project team aims to ensure that the drone's development aligns with sustainable engineering practices and minimizes potential harm to people and the environment [7].

The drone will be using a rechargeable lithium polymer battery to provide the most lightweight power supply possible. Some environmental impacts, like air pollution and land degradation, come with this decision [8]. While rechargeable lithium polymer batteries have become more popular with increased battery longevity, there are still many smaller consumer batteries that get disposed of all the time. The regulation of the disposal of lithium polymer batteries is lacking, leading to a continued negative environmental impact [9].

A further design impact involves electronic waste. The project's use of custom-made PCBs contributes to e-waste generation, as prototyping and development can result in discarded components. To minimize this impact, we will focus on being efficient with our PCB design and product purchases, as well as properly disposing of e-waste [10].

A design impact of this design is safety. Flying vehicles always offer some level of danger to those on the ground. To counteract this, we will strive to develop measures to prevent a rogue drone in the case of wireless signal disconnection. More specifically, many drones prevent this by simply immediately descending to the ground, in this case, or low battery voltage. We will also test the drone in open fields, as well as obtain the necessary certification for flying our drone [11]. We will be using resources such as Oregon State University's Division of Research

and Innovation, which gives us information on registering and getting necessary certifications [12].

The Solar-Glide Drone promotes sustainable drone technology by focusing on affordability and reusability. Socially, it encourages student engagement in renewable energy applications and responsible design. Economic factors include the usage of custom PCBs, Li-Po batteries, and other necessary components such as motors and a remote controller. Our goal is to balance cost-effectiveness with our part selection to ensure quality while keeping the price as low as possible.

The assessment highlights key environmental and safety impacts that we must consider throughout the design process. Responsible battery usage, e-waste management, and prioritizing safety are essential for minimizing the risks. Future work should include collaborations with battery recycling programs as well as e-waste management companies. By integrating sustainability and safety considerations into every stage of development, the Solar-Glide VTOL Drone can be a model example for socially responsible and sustainable engineering practices.

1.4. References

- [1] “espressif/esp-drone: Mini Drone/Quadcopter Firmware for ESP32 and ESP32-S Series SoCs,” GitHub Repository, GitHub. [Online]. Available: <https://github.com/espressif/esp-drone/tree/master/main>
- [2] “FreeRTOS — ESP32 API Reference,” ESP-IDF Programming Guide (stable), Espressif Systems, 2025. [Online]. Available: <https://docs.espressif.com/projects/esp-idf/en/stable/esp32/api-reference/system/freertos.html>
- [3] “Solar Panel Efficiency Explained: What It Is and Why It Matters in 2025,” NRG Clean Power, Jun. 18, 2025. [Online]. Available: <https://nrgcleanpower.com/learning-center/solar-panel-efficiency/>
- [4] “Global Consumer Drone Market Share Analysis Report, 2030,” Grand View Research Inc., San Francisco, CA, USA, Report ID 978-1-68038-831-2, 2023. [Online]. Available: <https://www.grandviewresearch.com/industry-analysis/consumer-drone-market>
- [5] G. Riney, “How Long Can a Drone Fly? In-Detail Guide to Understand Its Timing Better!”, Extreme Fliers, Aug. 06, 2025. [Online]. Available: <https://www.extremefliers.com/how-long-can-a-drone-fly/>
- [6] L. Dauphin, “Mapping the Amazon,” NASA Earth Observatory, 2018. [Online]. Available: <https://earthobservatory.nasa.gov/images/145649/mapping-the-amazon>

- [7] "1.4 principles of Sustainable Engineering," 1.4 Principles of Sustainable Engineering | EME 807: Technologies for Sustainability Systems,
<https://courses.ems.psu.edu/eme807/node/688>
- [8] B. A. J. 24, "The harmful effects of our lithium batteries," Greenly,
<https://greenly.earth/en-us/blog/industries/the-harmful-effects-of-our-lithium-batteries>
- [9] D. H. P. Kang, M. Chen, and O. A. Ogunseitan, "Potential Environmental and human health impacts of rechargeable lithium batteries in electronic waste," Environmental Science & Technology, <https://pubmed.ncbi.nlm.nih.gov/articles/PMC5920515/>
- [10] "About Us," Corvallis Sustainability Coalition,
<https://sustainablecorvallis.org/e-waste-disposal-in-the-corvallis-area/>
- [11] Recreational Drone Best Practices,
<https://www.oregon.gov/oprd/PRP/Documents/Drone-best-practices-20231023.pdf>
- [12] "Non-research operations," Division of Research and Innovation,
<https://research.oregonstate.edu/non-research-operations>

2. Top-Level Project Specification

The purpose of this section is to provide an overview of the top-level specifications of the Solar-Glide VTOL Drone project. A top-level and system-level box diagram are provided, showing the top-level architecture and how our blocks interact and the system-level showing only its interfaces derived from the engineering requirements in section 1.

This section contains in-depth descriptions of every block in the top-level architecture. Block functions, purposes, and component selections can be found within these sections.

An interface definition table is provided towards the end of this section. Each interface on the top-level architecture is included in this table. Specific values and ranges for each interface will be found here. These values are found from data sheets of the components we selected, research of values, and calculations.

2.1. Block Diagrams

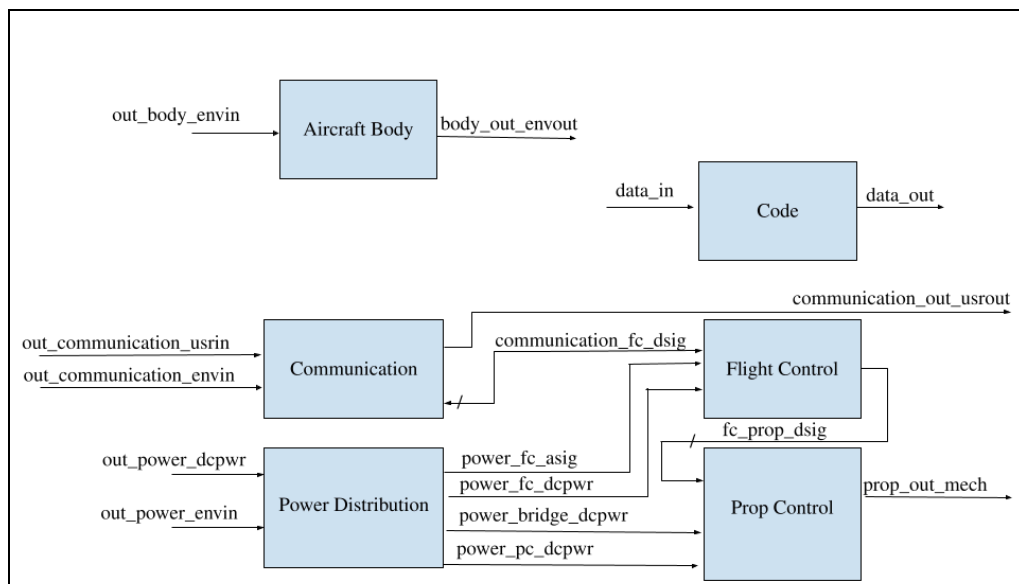


Figure x: Top-Level Architecture.

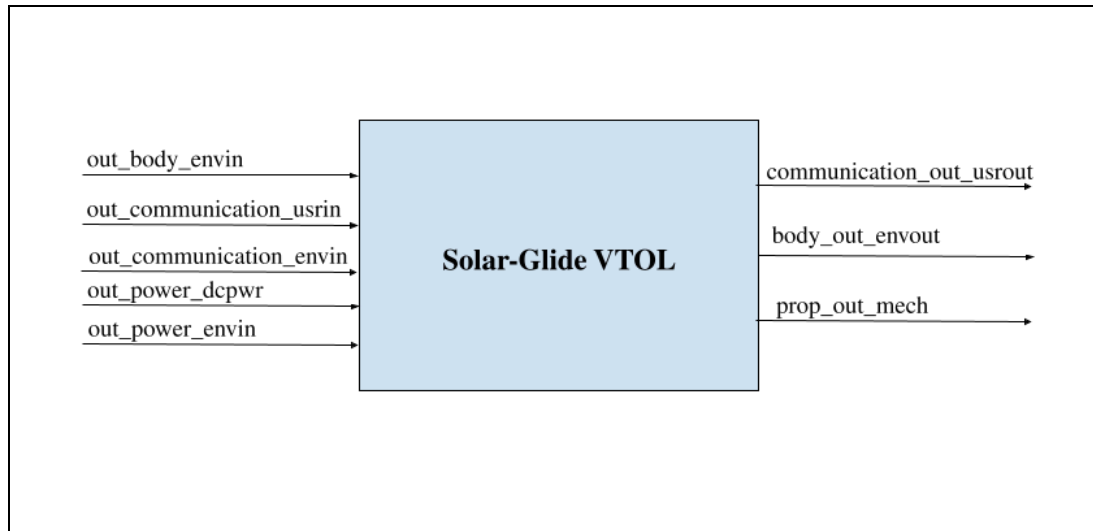


Figure x: System Level Black Box Diagram.

2.2. Block Descriptions

Prop Control

This block is a custom motor controller PCB that handles controlling the motors for the propellers on the drone. While the main microcontroller under Flight-Control handles multiple different high-level tasks like communication and navigation, the Prop Control requires quick and precise low-latency pulse-width-modulation (PWM) generation and real-time feedback handling from the motor sensors. To meet these timing requirements, the Prop Control includes its own onboard microcontroller, an Arduino nano [4], responsible for real-time motor drive operations, such as generating PWM signals for the gate drivers, current sensing to protect the motors from overcurrent or short circuits, and executing feedback control to ensure each motor maintains its desired speed and performance.

That onboard microcontroller will be driving five brushless DC motors, four 1350 KV motors are used for vertical lift, and one 900 KV motor provides forward thrust as a rear pusher propeller. Each vertical lift motor provides approximately 500g of thrust at around 40% throttle, requiring 24V and drawing about 4.5 A per motor [1]. This results in a combined draw of roughly 18A to lift the total 2000g weight of the drone. Once airborne, only partial throttle is required to maintain level flight because the motors no longer need to overcome static friction and ground effect drag during takeoff, and aerodynamic lift from the propellers becomes more efficient as the drone stabilizes in hover. Once the drone is high enough, the 5th rear propulsion motor will activate and the 4 vertical motors ramp down to 0A as they lock in place, while the 900KV rear pusher motor assumes propulsion, taking over to maintain forward flight. Steady flight would only require around 250g of thrust to overcome aerodynamic drag and sustain level motion, as the fixed-wing structure generates most of the lift. This corresponds to only about 2-3A of

continuous current draw at 24V during cruise [3]. Making cruise operations highly efficient compared to hovering.

Each motor will be powered by a dedicated integrated three-phase motor driver IC. Each driver consists of internal MOSFETs, a gate driver, current sensing, and protection circuitry. These ICs interface directly with the onboard microcontroller, which provides control signals that drive the internal gate drivers. The gate drivers then switch the IC's MOSFETs to regulate current flow from the battery to the motors. Power for the motor driver ICs is supplied directly from a 6S 22.V nominal and 25 peak charge 100C LiPo battery [2] through the power distribution block, while the motor controller controls timing and feedback for precise motor operation.

Communication

This system enables reliable exchange between the pilot and drone and supports manual control and live battery monitoring during flight. The RC link uses the remote controller and receiver pair to send pilot commands via channels over a 2.4 GHz radio connection. A channel refers to an independent control signal that is sent from the remote controller to the receiver. Each channel corresponds to a specific control input from the remote controller (Throttle, pitch, roll, yaw, etc). Onboard the drone, the FS-iA10B receiver outputs the pilots commands using the IBUS digital serial protocol which connects directly to the ESP32 flight control microcontroller via UART. The telemetry system provides battery voltage feedback to the pilot during flight. A FS-CVT01 voltage reader measures the batteries voltage and plugs into the receiver's sensor / telemetry port, allowing the receiver to relay that voltage information back to the remote controller for display to the pilot.

Code

The code block is responsible for interpreting sensor data, executing flight control algorithms, and managing communication between subsystems. It is implemented on a custom ESP32-based PCB and is inspired by several repositories. This includes an open source "arduino-drone-pid" code repository[9] and the "famous" esp-drone open source project [10]. This code is flashed on the custom ESP32 board to complement the flight-control hardware. It reads the 6 PWM signals given by the transceiver antenna [5], which corresponds to either direction or throttle of the motors. This will be heavily dependent on the method of the flight control algorithm.

Flight-Control

This block (and its corresponding PCB) serves as the central processing unit of the drone. It continuously gathers data from onboard sensors, executes PID control algorithms for regular motor control, and interfaces with the transceiver to both receive user commands and transmit real-time telemetry data, including battery status and velocity information. The sensor data, which is gathered for one, includes the IMU (Inertial Measurement Unit), more specifically the

MPU-6050 [6], which is low-cost and easily available. The IMU is essential for the quadcopter mode, to pitch, roll, and yaw smoothly by taking in accelerometer and gyroscopic data [7]. Now, for two, the barometer sensor is, in a way, the second most important for quadcopter flight, determining the altitude based on changes in air pressure to determine the altitude, and maintaining a consistent altitude [7]. More specifically, we will utilize the BMP280 [8], a low-cost barometer sensor for determining altitude. Each sensor will be tested, conventionally with breakout boards, before PCB integration. It is also worth mentioning that each sensor utilizes I2C [6][8]. Moving on, this block uses an x-bit bus digital signal that commands the prop control MCU (Arduino Nano), to throttle any of the 5 motors. Lastly, this block communicates with the transceiver to send and receive data over 6 channels [5].

Aircraft-Body

The aircraft body serves as the structural framework for the Solar-Glide VTOL Drone. It is designed to provide a balance between strength, flexibility, stability, and weight, ensuring the drone can operate in all modes of flight, which include vertical takeoff, fixed-wing flight, and fixed-wing glide. The body is designed with modular components that simplify maintenance and replacement (e.g., fastened wing and motor mounts). Internally, our electrical components will be housed in the fuselage, which aligns with our priority of creating a safe body to protect our sensitive components. This is implemented by a combination of 3D-printed springs, acting as shock absorbers for a floating PCB and internal foam padding. Vents are also included to prevent internal overheating of power electronics during extended use. Externally, any aircraft's body strength is highly dependent on its material composition. We opted for most of this to be 3D-printed PLA, with low infill whenever possible. Higher infill of ~40% for structural components (e.g., motor mounts or portions of fuselage) and lower infill of ~20% for aerodynamic surfaces (air foils). Furthermore, for ultimate weight saving, understressed components are made of foam whenever reasonably possible. All of this, designed in Fusion 360 CAD software, with possible future assistance of Aerodynamic simulation tools. The flexible photovoltaics do not constrain the shape/design of the aircraft, and easily conform to the wings.

Power Distribution

The power distribution block's purpose is to regulate and distribute different voltage levels to the components of the Solar-Glide VTOL Drone. A custom-made PCB board ensures each subsystem receives the correct voltage and current levels for proper operation.

A 6S 6000mAh 22.2V 100C Lipo battery powers the entire system [2]. The battery directly supplies power to the five 24V DC motors, four of which are DeepSpace RED LINE 2807 Brushless motors for the VTOL implementation [1], and one GEPRC EM3115 3115 900KV 6S Brushless motor for forward-flight [3]

Our system contains lower-voltage electronics, and step-down voltage regulators are used to power these components. The 22.2V provided by the battery is converted to 5V, which powers

our custom-made ESP32 board. This rail also provides power to two servos, which operate at 5V. Protection features, such as decoupling capacitors and TVS diodes, are implemented on the board to safeguard our components against voltage spikes and current surges.

2.3.Interface Definitions

Name	Properties
Prop_out_mech: Output	- Steel Shaft = 5mm +- 0.3mm - Motor mount screws: 3mm +- 0.3mm - Propellor = 7037 - Connector = MR60PW
receiver_controller_rf: Output	- Protocol: AFHDS 2A - Expected Operating Range > 50m - Data carried Battery voltage +- 1% error
controller_reciever_rf: Input	- Protocol : AFHDS 2A - Expected operating range > 50 - Channel count supported: 4
out_power_dcpwr	- Inominal: 50 A - Ipeak: 236.6A - Vmax: 25.2V - Vmin: 18V
out_power_envin	- env_power_voltage_max = 25 V - env_power_current_max = 2.0 A - env_power_power_max = 30 W
power_fc_asig	- Vrange: 0-25.2V - Minimum Sampling Frequency: >= 1 Hz - Frequenzy: 0 Hz - Read Resolution: 0.75V
power_pc_dcpwr	- Inominal: 24.56 mA - Ipeak: 27.68 mA - Vmax: 2.4V - Vmin: 3.6V
power_fc_dcpwr	- Inominal: 50 mA - Ipeak: 100 mA - Vmax: 3.45V - Vmin: 3.10V

power_bridge_dcpwr	- Inominal: 10 A - Ipeak: 38.1 A - Vmax: 25.2V - Vmin: 18V
prop_out_mech	- Vertical motor KV (RPM per Volt): 1350 KV - Rear propellor KV (RPM per Volt): 900 KV - Vertical propellers size: 7 inch - Rear propellor size: 10 inch
data_in	- Altitude_m - bmp.temperature (F) - Bool readBNO085 (variables: rc_roll, rc_pitch, rc_yaw) (-90-90 degrees)
data_out	- Code_motors_cmd (variables: motor_cmd_qm1...qm4, motor_cmd_gm) - code telemetry_frame (variables: out.temperatureF) - Function: float PWM_PW_To_DutyPercent(uint16_t us)
out_body_envin	- Motor_Q-Mount Bolt: [M3 * 8] x 16 - Motor_Q-Mount Mounting holes: [M3.1±0.1] x 16 - Mount-Boom Bolt: [M4 * 16] x 16 - Mount-Boom Mounting holes: [M4.1 ±0.1] x 16
body_out_envout	- Horizontal stabilizer chord length: 95.4 ± 3.5 mm - Horizontal wing spar length: 0.4 m. - Boom length: 1 ± 0.10 m

Verification process (for system engineering requirements)

1. The system must glide at least 4 meters.

Tools:

- Tape measure.

Setup:

- A. Extend a tape measure straight along the ground (preferably indoor gym, otherwise calm day).
- B. Mark the 0 m point clearly.
- C. Flash board with test script #1 (initial delay, then pusher prop at 100% throttle for x seconds).

Test:

- A. From the 0 m mark, give the drone a consistent forward throw parallel to the tape measure (this replaces the initial velocity the quad motors would normally provide).
- B. Let it fly until the first touchdown and mark that touchdown point on the tape measure.
- C. Pass if touchdown distance is ≥ 4.0 m.

2. The system shall recharge its onboard batteries from a nominal 120 V AC, 60 Hz single-phase supply compatible with standard North American plugs.

Tools:

- System battery charger
- 120 V AC, 60 Hz single-phase wall outlet (NEMA 1-15/5-15)
- Multimeter
- Timer

Setup:

- A. Disconnect battery EC5 connector from system.
- B. Connect EC5 connector and battery cell connector to the system battery charger.
- C. Plug the system battery charger into a 120 V AC, 60 Hz single-phase wall outlet.
- D. Measure and record the battery pack voltage with a multimeter before charging .

Test:

- A. Start charging.
- B. Wait 5 minutes.
- C. Measure and record the battery pack voltage with a multimeter after waiting
- D. Pass if $V_{\text{new}} > V_{\text{old}}$, as the battery now should hold a greater charge.

Video evidence of the battery charging can be found below:

[Battery ER Setup](#)

[Battery ER Timelapse](#)

[Battery ER Results](#)

3. The system must be controlled wirelessly from at least 50 meters.

Tools:

- A spotter.

Setup:

- A. Go to a football field, and set the system down at the 55 yard mark on the field (greater than 50 meters).
- B. Open drone hatch to view SMD Green LED is ON, leaving the hatch open.
- C. Have the spotter stand by the drone, while the pilot stands at the 0 yard line with the system remote.

Test:

- A. Pilot applies any joystick command.
- B. Spotter verifies LED flashes.

A set of videos for verification can be found below:

[Measuring Distance for ER](#)

[ER Video](#)

[Alternate Full View Video of ER](#)

4. The system must include independent charging.

Tools:

- Multimeter
- Timer

Setup:

- A. Connect solar charging to the battery.
- B. Measure and record battery pack voltage before charging (V_{old}).

Test:

- A. Place the solar panel in sunlight.
- B. Wait 10 minutes.
- C. Measure battery voltage V_{new} , and verify $V_{new} > V_{old}$.

5. The system must remain airborne for at least 15 minutes on a single charge.

Tools:

- Stopwatch

Setup:

A. Be in an open, flat area with no obstacles and away from any people.

Test:

A. Start the stop watch once the drone leaves the ground.

B. Fly the drone upwards and cruise around for 15 minutes. Hovering is also acceptable as long as the drone is continuously airborne for 15 minutes.

C. At 15 minutes, verify the drone is still airborne.

D. Land the drone and stop the stopwatch.

6. The system shall vertically ascend to an altitude of at least 4 meters above ground level.

Tools:

- 14 foot long string (14 feet = 4.2672 meters).

- Tape

Setup:

A. Be in an open, flat area with no obstacles and away from any people.

B. Tape the 14 foot long string to the bottom center of the drone.

Test:

A. Fly the drone upwards and high enough such that the string is at least fully extended and the bottom of the string is off the ground.

B. Hover for at least 5 seconds.

C. Confirm the string is still fully extended and off the ground, then land the drone.

7. The system must relay the battery charge back to ground.

Tools:

- Multimeter

- Calculator

Setup:

A. Inside the drone, ensure the voltage reader (FS-CVT01) is connected to the battery terminal.

B. Inside the drone, ensure the receiver (FS-iA10B) is powered.

C. Have the remote controller (FS-i6X), turn it on.

D. Flash a test script to the flight controller (ESP32) that does nothing. We do not want any motors / propellers spinning.

Test:

A. On the remote controllers display, identify the voltage reading from the FS-CVT01 and record this voltage as Vread.

B. Measure the battery voltage at the battery terminal using a multimeter and record this voltage as Vbat.

C. Verify the Vread voltage is approximately equal to the Vbat voltage.

8. The system must be lightweight with a net weight of less than 6000 grams.

Tools:

- Bathroom scale
- Calculator

Setup:

A. Assemble the drone in ready-to-fly configuration (battery installed, all electrical components inside, motors attached).

B. Place the bathroom scale on a hard, flat surface.

C. Turn on the scale and confirm it reads 0.

Test:

A. Measure a single person only:

1. Stand centered on the scale.
2. Record reading as weight1.

B. Measure a single person + drone:

1. Stand centered on the scale while holding the drone.
2. Record reading as weight2.

C. Calculate drone mass:

1. Drone mass = weight2 – weight1.

D. Repeat steps A–C 1–3 times. Use the average drone mass as the final result.

E. Pass if average drone mass < 6000 g.

2.4.References

[1] "Deepspace Red Line 2807 brushless motor 1350KV 6s for FPV 7-8inch freestyle long range drones-1350KV," NewBeeDrone, <https://newbeedrone.com/products/deepspace-red-line-2807-brushless-motor-1350kv-6s-for-fpv-7-8inch-freestyle-long-range-drones-1350kv> (accessed Oct. 26, 2025).

[2] "CNHL racing series 6200mAh 22.2V 6s 90c lipo battery with EC5 plug," ChinaHobbyLine, <https://chinahobbyline.com/collections/cnhl-voltage-22-2v-6s-lipo-batteries/products/cnhl-racing-series-6200mah-22-2v-6s-90c-lipo-battery-with-ec5-plug> (accessed Oct. 26, 2025).

[3] Banggood.com, "GEPRC EM3115 3115 900KV 6s brushless motor for 8 9 10 Inch long range FPV RC racing drone," [www.banggood.com](https://www.banggood.com/GEPRC-EM3115-3115-900KV-6S-Brushless-Motor-for-8-9-10-Inch-Long-Range-FPV-RC-Racing-Drone-p-2003873.html?gmcCountry=US¤cy=USD&cur_warhouse=CN&createTmp=1&ID=629197049289), https://www.banggood.com/GEPRC-EM3115-3115-900KV-6S-Brushless-Motor-for-8-9-10-Inch-Long-Range-FPV-RC-Racing-Drone-p-2003873.html?gmcCountry=US¤cy=USD&cur_warhouse=CN&createTmp=1&ID=629197049289 (accessed Oct. 26, 2025).

[4] Arduino Nano ESP32 product page short description, https://www.mouser.com/datasheet/2/34/Arduino_07202023_Nano_ESP32_Product_Page_Overview-3240096.pdf (accessed Oct. 27, 2025).

[5] Flysky RC Model Technology Co., Ltd., FS-i6X Digital Proportional Radio Control System User Manual, Edition 2016-08-17, 34 p. [Online]. Available: <https://static1.squarespace.com/static/5bc852d6b9144934c40d499c/t/5d6f66305487dc0001e34939/1567581800555/FS-i6X+User+manual+20160817.pdf>

[6] "Oregon State University EECS Inventory Datasheet P1462469087," School of Electrical Engineering & Computer Science, Oregon State University, Corvallis, OR, [Online]. Available: https://eeecs.engineering.oregonstate.edu/education/inventory_datasheets/P1462469087.pdf

[7] "What Sensors Are Installed in a Drone?," Airclip Guide, Airclip Service GmbH & Co. KG, Dresden, Germany, [Online]. Available: <https://www.airclip.de/What-sensors-are-installed-in-a-drone>

[8] Bosch Sensortec, BMP280 Digital Pressure Sensor Datasheet, Document No. BST-BMP280-DS001-11, Rev. 1.14, May 5 2015. [Online]. Available: <https://cdn-shop.adafruit.com/datasheets/BST-BMP280-DS001-11.pdf>

[9] 0xekez, "arduino-drone-pid: PID for controlling a DIY Arduino drone," GitHub, <https://github.com/0xekez/arduino-drone-pid>

[10] Espressif Systems, "esp-drone: Mini Drone/Quadcopter Firmware for ESP32 and ESP32-S Series SoCs," GitHub, 2024. [Online]. Available: <https://github.com/espressif/esp-drone>

[11] Lipo battery voltage, discharge rate and cycle life | Grepow, <https://www.grepow.com/blog/basis-of-lipo-battery-specifications.html> (accessed Nov. 2, 2025).

3. Block Designs

3.1. Prop Control

This block contains a DIY ESC (Electronic speed controller) created by Noah Amack. There will be 5 ESCS in total, one for each motor on the drone, but this document focuses on the design of a single ESC.

3.1.1.Description

This ESC is built around an STM32F030F4P6 microcontroller, which receives a PWM throttle signal from the ESP32 Flight controller. The STM32 interprets this signal and generates the corresponding PWM outputs to drive a half-bridge gate driver circuit. These gate drivers, in turn, control the MOSFETS in the three-phase full bridge, providing power to the brushless motor. The brushless motors are designed with 3 phases (coils), referred to as U,V, and W phases. The three-phase full bridge MOSFET circuit initially allows current flow through two phases, while one phase remains off. This creates a rotating magnetic field that spins the motors rotors. When the rotor passes the coil of the phase that is off, a voltage known as back EMF is induced.

The back-EMF is essential for sensorless motor control, as it provides the rotors position without the need for physical sensors. In this design the LM339 comparator circuit monitors the voltages of the three phases (U,V, and W) and compares it against a common reference voltage called V-COM (voltage common). When the back EMF of a phase crosses V-COM, the comparator outputs a digital signal to the STM32 signaling a zero-crossing. At this zero crossing the STM32 microcontroller reads these signals and then produces a precise switching sequence of the three phase full bridge. During each sequence, one phase is driven high, one is driven low and the third is left floating to sense the back-EMF. The STM32 continuously switches which phases are high, low, or floating to maintain a rotating magnetic field in the stator. This switching allows the motors to spin the propellers smoothly and efficiently.

3.1.2.Design

This block receives a PWM signal (dsig) from the flight controller, which the onboard STM32 microcontroller uses to drive the MOSFETS. This allows current from the power distribution block to flow to the motors, spinning the propellers to produce thrust.

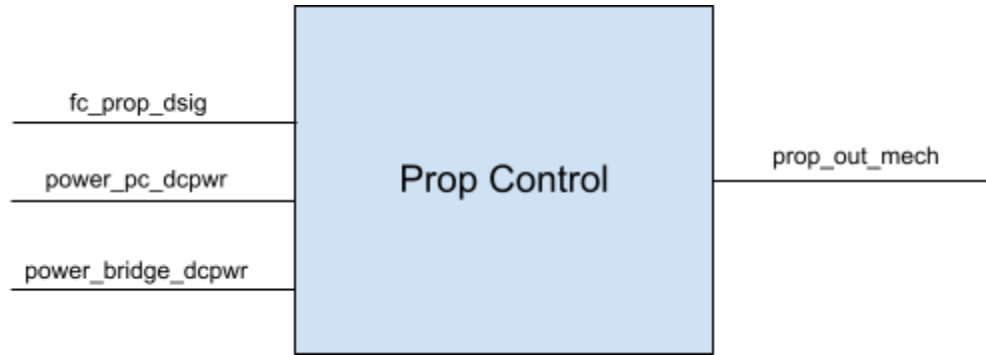


Fig.1: Black Box Diagram

This schematic consists of a DIY ESC built around an onboard STM32F030F4P microcontroller. The microcontroller generates a PWM control signal for three UCC27301ADR half bridge gate drivers, which in turn drive the CSD18533Q5A MOSFETs in the three phase bridge configuration. . There is a Filter network to reduce noise in the phase signals. The LM3390 comparators reads these smoother signals to detect the back EMF from each phase, allowing the microcontroller to keep the motor spinning.

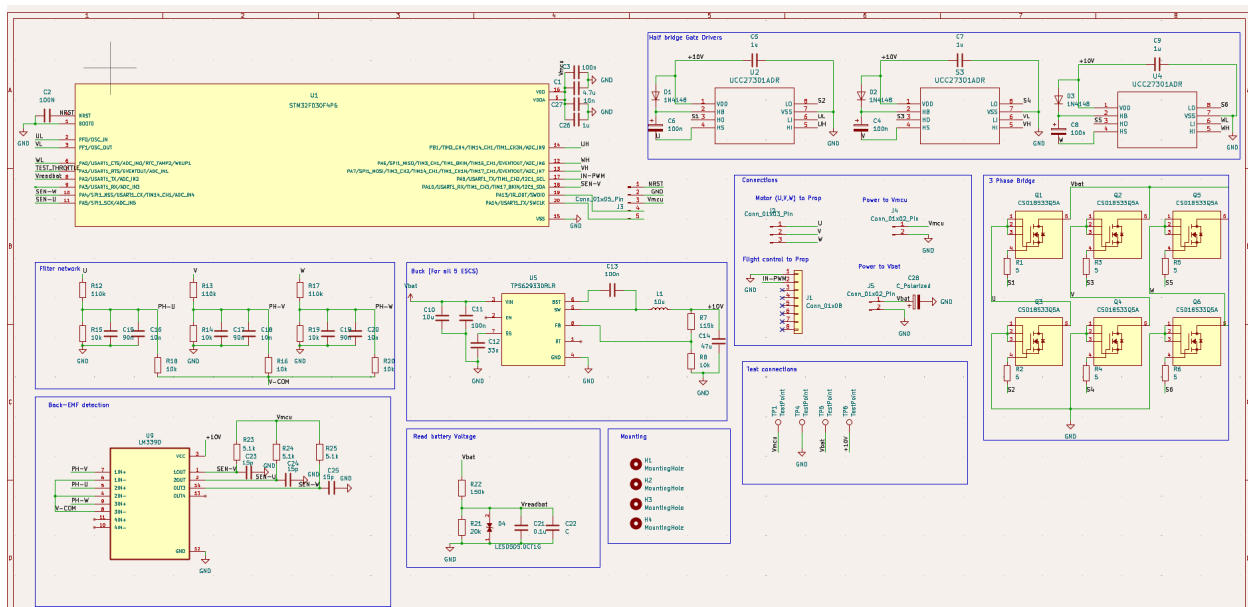


Fig. 2: Schematic for ESC

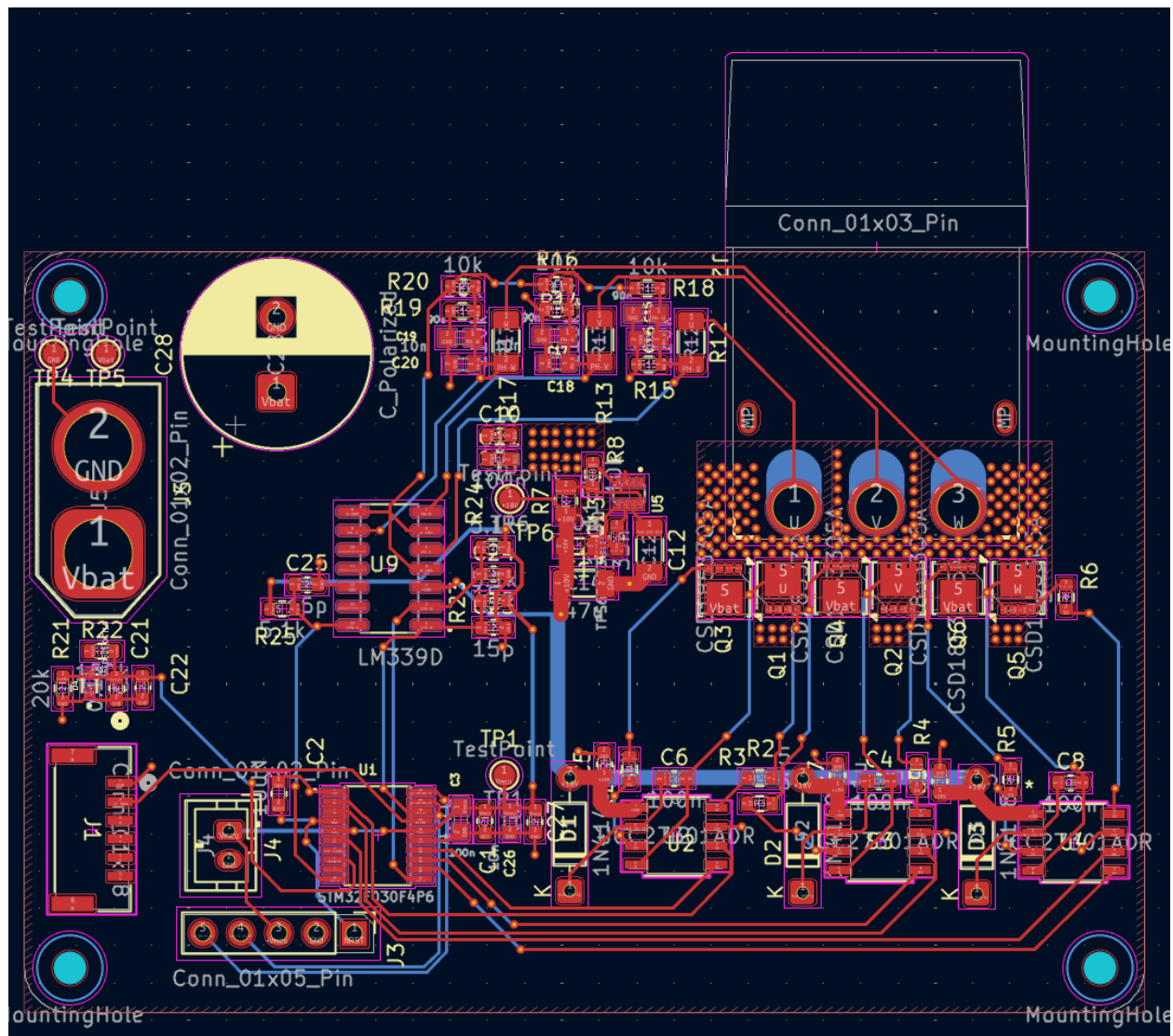


Fig. 3: PCB for ESC

3.1.3. General Validation

The selected microcontroller, MOSFETS, and gate drivers meet the speed, voltage and current requirements of the propulsion system. The STM32F030F4P6 operates with low latency and sufficient PWM frequency for smooth motor commutation. It will be supplied its 3.3V operating voltage by a buck converter from the power distribution block. A 6S 6200 mAH Lipo battery will be connected to the three phase bridge to supply power to the motors so the MOSFETS need to be rated at a high voltage and current. The CSD18533Q5A MOSFETS in the three phase full bridge are capable of high peak currents (100A) and a high VDS (Drain to Source voltage) of 60V, ensuring reliable MOSFET switching even at

high duty cycles. The MOSFETS require a 10V VGS (Gate to source voltage) to be on and the most efficient, which an onboard buck converter will provide to the UCC27301ADR half bridge gate drivers to drive the gate voltages of the MOSFETS.

Electrical noise migration is handled through the filter network which will ensure clean phase-voltage readings so the LM339 comparators accurately detect zero crossings, allowing for reliable rotor position estimation and smooth phase sequence switching.

This ESC layout is designed for ease of assembly and maintenance. All components are standard, readily available parts that can be sourced globally, enabling low-cost reproduction of multiple ESC units. The organized schematic layout separating power regulation, signal processing, and phase control simplifies the circuit for debugging and replacement.

3.1.4.Interface Validation

Interface property		
*Be sure to use the naming convention “from_to_type” here.	Why is this interface property this value?	How do you know your design details will meet or exceed this property? Cite your sources in IEEE.

power_pc_dcpwr

Vmin = 2.8V	This is the minimum operating voltage for the STM32F030F4P6 according to the datasheet[1, p.1] plus some additional margin to be safe.	The 6S rated nominal 22.2 V power supply will be regulated to 3.3V through a buck converter[10], ensuring that the STM always operates above the minimum operating voltage.
Vmax = 3.5V	This is 1V below the maximum operating voltage for the STM32F030F4P6 according to the datasheet[1, p.1], to ensure safe operating conditions.	The 6S rated nominal 22.2 V power supply will be regulated to 3.3V through a buck converter[10], ensuring that the STM always operates below the maximum operating voltage.
Inominal = 2.77mA	The nominal was calculated with the typical supply current (26mA) in run mode when flashed and supplied with 3.6V, according to the datasheet [1, p.50]. There will be 3.3V supplied:	The 6S rated nominal 22.2 V power supply will be regulated to 3.3V through a buck converter[10], ensuring that the system operates at the nominal current.

	$I_{3.3\text{nom}} = I_{3.6\text{nom}} \times 3.3/3.6$ $= 3\text{mA} \times 3.3/3.6 = 2.77\text{mA}$	
$I_{\text{peak}} = 27.68\text{mA}$	The nominal was calculated with the typical supply current (30.2 mA) in run mode when flashed and supplied with 3.6V, according to the datasheet [1, p.47]. There will be 3.3V supplied: $I_{3.3\text{max}} = I_{3.6\text{max}} \times 3.3/3.6$ $= 30.2\text{mA} \times 3.3/3.6 = 27.68\text{mA}$	The 6S rated nominal 22.2 V power supply will be regulated to 3.3V through a buck converter[10], ensuring that the system operates below the peak current.

power_bridge_dcpwr

$V_{\text{min}} = 18\text{V}$	This is the minimum safe operating voltage of a 6S Lipo battery corresponding to 3.0V per cell, which is the typical lower limit before cell degradation occurs [2].	The ESC's MOSFETS are rated for 0 to 60V according to the datasheet [5, [p.1], which will allow the system to remain fully functional at the minimum 18V input. The gate driver ensures reliable switching and conduction near this lower voltage.
$V_{\text{max}} = 24\text{ V}$	A fully charged 6S Li-po battery reaches 25.2V (4.2 V per cell), 24V is used as the standardized operating voltage in the motor manufacturer's test data[4]. Therefore, 24V is adopted as the system's V_{max}.	The ESC's MOSFETS are rated for a VDS (Drain to source voltage) of 60V [5, p.1], which provides over 2x a safety margin above the maximum battery voltage, ensuring safe operation.
$I_{\text{nominal}} = 1.4\text{ A}$	Continuous operating current drawn by a single motor at the nominal throttle setting during bench testing. This current represents the steady-state load expected during typical operation.	To determine I_{nominal}, the ESC was powered from a bench power supply set to 24V and current limited to 3A. The motor was driven at the test throttle, and the PWM duty cycle was adjusted as needed to maintain the full 24V without the supply entering current-limit mode. Under these conditions, the measured steady-state current was 1.4A. This measured 1.4A nominal

		current is well below the continuous current rating of the MOSFETS [5, p.1].
$I_{peak} = 43A$	Current possibly drawn by a single motor at the peak operating throttle according to the motors datasheet[4].	Based on motor test data, at 100% throttle the current draw is 38.1A with a possible spike up to 43A [4] which is within the current handling capability of the ESC's MOSFETS (rated for 100A) [5, p.1].

fc_prop_dsig

Logic High = 3.3V	The flight controller (ESP32) operates at 3.3V [6, p.29], so its GPIO high level is equal to its supply voltage.	The ESC MCU (STM32F030F4P6) correctly interprets any $input > 0.7 \times VDD$ as high [1, p.61], where $VDD = 3.3V$. So $Logic\ high > 0.7 \times 3.3 = 2.31V$ is valid. The Flight control MCU logic high 3.3V is well above that.
Logic Low = 0V	The flight controller (ESP32) drives 0V for logic low states.	The ESC MCU (STM32F030F4P6) correctly intercepts any $input > 0.3 \times VDD$ as low [1, p.61], where $VDD = 3.3V$. So $Logic\ low > 0.3 \times 3.3 = 1V$ is valid. The Flight control MCU logic low 0V is well below that.
PWM frequency = 400 Hz	This control frequency refers to how often the flight controller (ESP32) sends throttle commands to the ESC. In PWM based controlled systems, this frequency is typically between 50 and 490 Hz in modern implementations [7]. 400 Hz is	The PWM control frequency will be generated by configuring the ESP32 timer. $f_{PWM} = f_{LED_CLK} / (LEDC_CLK_DIV \times 2^{LEDC_TIMER \times DUTY_RES})$ [8, p.1313]. It is determined by the 80MHZ peripheral clock using configurable settings. This allows PWM frequencies from below 1Hz

	a common PWM frequency to use.	up to MHz, well beyond the 50-490 Hz range required for the ESC control signals.
Duty cycle = 40 to 80%	The duty cycle defines the percentage of time the PWM signal remains high within one period. For throttle control this value determines how much power is delivered to the motor. A full range from 40% (motor off) to 80%(full throttle) is chosen to provide a linear relationship between the PWM signal and the motor speed command.	The ESP32 microcontroller supports hardware PWM generation through the LEDC peripherals [6, p.25], allowing precise duty cycle control from 0 to 100%.

prop_out_mech

Steel Shaft: 5mm +- 0.3mm	The manufacturer designed the motor with a 5mm hardened steel shaft [4] to allow attachment of a 7037 prop.	During bench testing a 7037 propeller was attached to the steel shaft.
Motor mount screws: 3mm +- 0.3mm	The manufacturer pre installed a motor mount with m3 screws[4] to allow proper installation of the motor.	During bench testing the mount was attached to a DIY motor stand via the m3 screws.
Propellor = 7037	The propeller designation "7037" indicates a 7 inch diameter and 3.7 inch pitch. The diameter is the distance across the circle traced by the blade tips while the pitch is the theoretical distance the propeller would move forward in one revolution	The GF7037 propeller dimensions are specified by the manufacturer and validated through their aerodynamic performance testing[4]

Connector = MR60PW	The motor interface uses an MR60PW connector. A MR60PW male connector is mounted on the PCB, and an MR60PW female connector is mated to it. The motor phase wires are soldered to the female connector, providing a removable and mechanically secure power connection between the ESC and motor.	The MR60PW connection can withstand 30A continuous current [11], ensuring proper operations of the motor. Connector mating and high current operation of the MR60PW connector pair were confirmed during bench testing.
--------------------	---	---

3.1.5. Verification Process (For one ESC and one motor)

Verify power_logic_dcpwr

- Connect the STM32 microcontroller to the power distribution block or an external regulated DC power supply. This can be done via the JST connector.
- Flash the test script ESC_Test_OneSeq24V.
- For each test, keep power flowing for around 10 seconds to ensure stable voltage and current readings.
- Set the power supply to the minimum voltage (2.8V).
 - Confirm that the system works (motor spins).
 - Observe and ensure that the current drawn is around the nominal current and below the peak current.
- Set the power supply to the maximum voltage (3.5V)
 - Confirm that the system works (motor spins).
 - Observe and ensure that the current drawn is around the nominal current and below the peak current.
- Set the power supply to the nominal voltage (3.3V), representing the actual system operating condition.
 - Confirm that the system works (motor spins).
 - Observe and ensure that the current drawn is around the nominal current and below the peak current.

Verify power_motor_dcpwr

- Connect the XT-60 connector on the ESC to the power distribution block or an external regulated DC power supply.
- For each test, keep power flowing for around 10 seconds to ensure stable voltage and current readings.
- Flash the test script ESC_Test_OneSeq18V
 - Set the power supply to the minimum voltage (18V).
 - Confirm that the system works (motor spins).

- Observe and ensure that the current drawn is around the nominal current and below the peak current.
- Flash the test script ESC_Test_OneSeq24V
 - Set the power supply to the maximum voltage (24V)
 - Confirm that the system works (motor spins).
 - Observe and ensure that the current drawn is around the nominal current and below the peak current.

Verify fc_prop_dsig

- Connect the flight controller (ESP32) PWM output pin to the ESC signal input pin on the STM32.
- Power on both the flight controller and ESC using a regulated 3.3V supply.
- Verify logic levels:
 - Use an oscilloscope or logic analyzer to measure the PWM signal from the ESP32.
 - Confirm that the logical high level is approximately 3.3V and the logical low is 0V.
- Verify PWM frequency:
 - Using the oscilloscope, measure the PWM signal frequency from the ESP32.
 - Confirm that the frequency is approximately 400Hz.
 - Verify that the ESC input signal remains stable.
- Verify duty cycle range:
 - Connect the logic analyzer with a USB.
 - Attach the female side of a male-to-female jumper cable to any logic-analyzer channel, and connect ground with a female-to-female cable between the analyzer and the PCB.
 - Start the capture in the logic-analyzer software, then probe each test pad next to the six-pin connector for at least one second using the male end of the jumper cable.
 - In the software, confirm the frequency is 400 Hz. One pad should show D = 40, and another should show D = 80.

Verify the prop_out_mech

- Verify propellor dimensions
 - Measure the propeller diameter with some type of measuring tool to confirm the GF7037 specification (7 inch diameter)
- Verify connector
 - Check if the connector MR60PW can connect from the board to the motor
- Verify motor mount screws
 - Measure the motor mount screws and verify 3mm diameter.
- Verify motor shaft
 - Measure the motors shaft and verify the diameter is 5mm.

3.1.6. References

[1] St,

<https://www.st.com/content/ccc/resource/technical/document/datasheet/a4/5d/0b/0e/87>

</c4/4d/71/DM00088500.pdf/files/DM00088500.pdf/jcr:content/translations/en.DM00088500.pdf> (accessed Nov. 2, 2025).

- [2] Lipo battery voltage, discharge rate and cycle life | Grepow,
<https://www.grepow.com/blog/basis-of-lipo-battery-specifications.html> (accessed Nov. 2, 2025).
- [3] "CNHL racing series 6200mAh 22.2V 6s 90c lipo battery with EC5 plug," ChinaHobbyLine,
<https://chinahobbyline.com/collections/cnhl-voltage-22-2v-6s-lipo-batteries/products/cn-hl-racing-series-6200mah-22-2v-6s-90c-lipo-battery-with-ec5-plug> (accessed Nov. 2, 2025).
- [4] "Deepspace Red Line 2807 brushless motor 1350KV 6s for FPV 7-8inch freesty long range drones-1350KV," NewBeeDrone,
<https://newbeedrone.com/products/deepspace-red-line-2807-brushless-motor-1350kv-6s-for-fpv-7-8inch-freestyle-long-range-drones-1350kv> (accessed Oct. 26, 2025).
- [5] CSD18533Q5A Texas Instruments | Discrete Semiconductor products | DigiKey,
<https://www.digikey.com/en/products/detail/texas-instruments/CSD18533Q5A/3674878> (accessed Nov. 2, 2025).
- [6] Espressif Systems, "ESP32-S3-WROOM-1 & WROOM-1U Datasheet (v1.5)", Doc. No. ESP32-S3-WROOM-1_WROOM-1U_Datasheet_EN, 4.0 years (≈2021) old. [Online]. Available:
https://www.espressif.com/sites/default/files/documentation/esp32-s3-wroom-1_wroom-1u_datasheet_en.pdf
- [7] G. Staff, "What is an Electronic Speed Controller & How does an ESC work," Tyto Robotics,
<https://www.tytorobotics.com/blogs/articles/what-is-an-esc-how-does-an-esc-work#:~:text=The%20control%20frequency%20refers%20to.reduce%20latency%20in%20the%20system.&text=The%20switching%20frequency%20refers%20to.it%20at%20the%20switching%20frequency>. (accessed Nov. 2, 2025).
- [8] ESP32-S3,
https://www.espressif.com/sites/default/files/documentation/esp32-s3_technical_reference_manual_en.pdf (accessed Nov. 3, 2025)
- [9] MrSottong and Instructables, "Made your own ESC brusless motor: STM32F030," Instructables,
<https://www.instructables.com/Made-Your-Own-ESC-Brusless-Motor-STM32f030/> (accessed Nov. 2, 2025).

[10] AP63200/AP63201/AP63203/AP63205 document number: DS41326 rev. 3 - 2 1 of 18, <https://www.diodes.com/assets/Datasheets/AP63200-AP63201-AP63203-AP63205.pdf> (accessed Nov. 3, 2025).

[11] Aliexpress, <https://www.aliexpress.us/item/3256803512098703.html?gatewayAdapt=glo2usa4itemAdapt> (accessed Dec. 3, 2025).

3.2. Communication

The communication block consists of a FS-i6X remote controller, a FS-iA10B receiver, and a FS-CVT01 voltage reader.

3.2.1. Description

This system enables reliable exchange between the pilot and drone and supports manual control and live battery monitoring during flight. The RC link uses the remote controller and receiver pair to send pilot commands via channels over a 2.4 GHz radio connection. A channel refers to an independent control signal that is sent from the remote controller to the receiver. Each channel corresponds to a specific control input from the remote controller (Throttle, pitch, roll, yaw, etc). Onboard the drone, the FS-iA10B receiver outputs the pilots commands using the IBUS digital serial protocol which connects directly to the ESP32 flight control microcontroller via UART. The telemetry system provides battery voltage feedback to the pilot during flight. A FS-CVT01 voltage reader measures the batteries voltage and plugs into the receiver's sensor / telemetry port, allowing the receiver to relay that voltage information back to the remote controller for display to the pilot.

3.2.2. Design

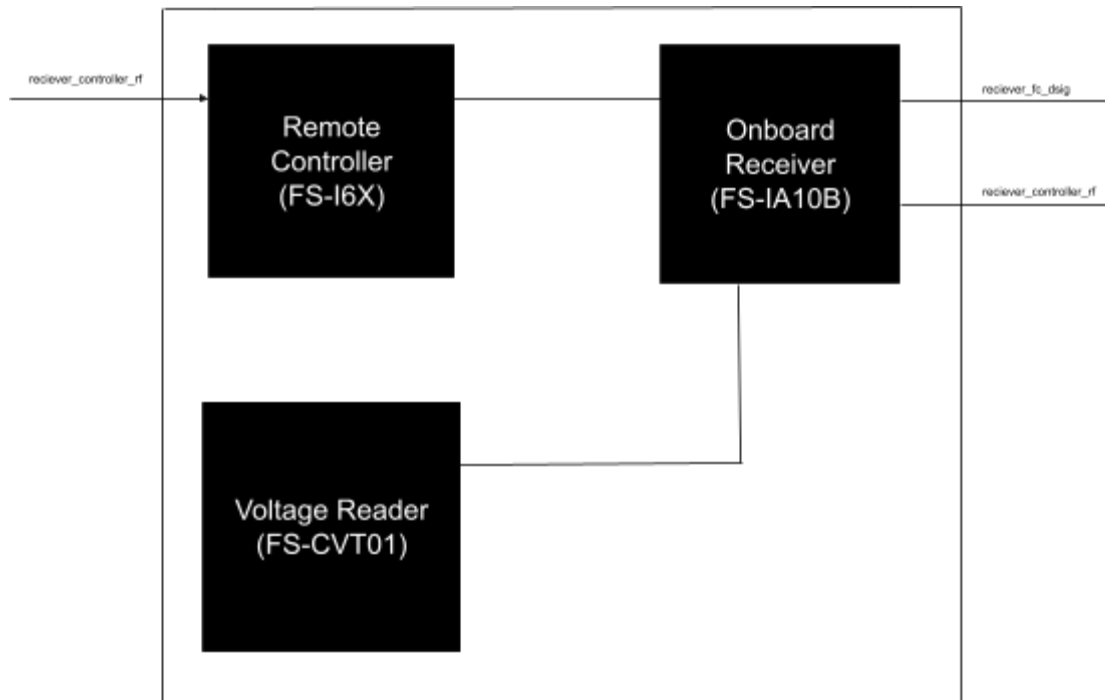


Figure 1: Communication Block Diagram

This diagram (figure 2) displays the connections of the communication system. There is a UART connection between the ESP32's Rx pin and the FS-IA10B receiver's output pin, allowing the receiver to output data to the flight controller utilizing IBUS digital series protocol. Another connection is between the FS-CVT01 and the receiver to allow the receiver to relay battery voltage back to the ground.

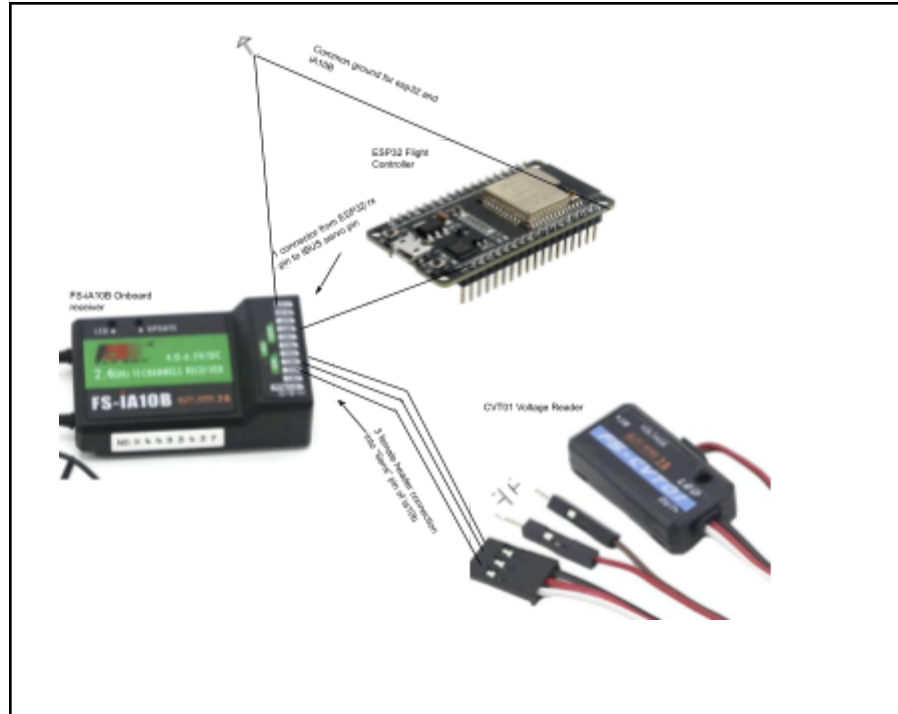


Figure 2: Communication Connection Drawing:

3.2.3. General validation

This block directly satisfies the system requirement that the drone must be controlled wirelessly from at least 50 meters. The FS-i6X transmitter and FS-iA10B receiver form a 2.4GHz RC link using FlySky's AFHDS 2A protocol designed for long range line-of-sight operation[1][3]. The FS-iA10B is specified for approximately 500-1500m range under ideal conditions[3], which provides a significant margin above the 50m minimum requirement. Therefore even with environmental factors and antenna placement constraints, this RC link provides enough wireless control capability for the system's intended operating distance. This block also satisfies the requirement that the system must relay battery charge back to the ground. The FS-CVT01 voltage reader measures the onboard battery voltage and connects to the receiver's sensor/telemetry (SENS) port, allowing the receiver to transmit that battery voltage information back to the FS-i6X transmitter for live display[3][4].

In addition to meeting the system requirements, the communication design supports safe and practical drone operation by ensuring the flight controller receives pilot inputs through a single, consistent digital interface. The iBUS protocol operates through a single-wire UART interface at 115200 baud[5][7], allowing all of the control channels received from the remote controller to be transmitted digitally over one single line, rather than using the common alternative of multiple PWM outputs per channel. This reduces the amount of pins required on the drones flight controller (ESP-32), minimizing system complexity while ensuring stable and reliable communication. The iBusBM library specifically supports ESP32 UART hardware[5], confirming compatibility with our selected flight controller. Additionally, utilizing the FS-CVT01 voltage

reader to transmit data back to the remote controller instead of using an extra RX and TX pin on the ESP32, further reduces the complexity of the hardware and software.

3.2.4. Interface Validation

reciever_fc_dsig: Output

Protocol: UART IBUS	The FS-iA10B outputs pilot command channels over FlySky's iBUS, which is varied over a hardware UART serial interface.	The iBusBM library documentation describes iBUS as a FlySky protocol that runs on a single digital line using UART[5].
Datarate: 115200 baud	iBUS uses a UART link that runs at 115200 baud [5][7].	The iBusBM library documentation states that the iBUS UART baud rate will be fixed at 115200 baud. The library is compatible with the ESP32 UART[5].
Logic Level: 3.1~3.45 V	ESP32 UART Rx is 3.3V logic, so the system must ensure incoming iBUS signal does not exceed that.	A multi meter test was conducted which confirmed that the signal pin of the FS-iA10B outputs 3.3V. Ensuring the ESP32 will receive current logic level inputs.

receiver_controller_rf: Output

Protocol: AFHDS 2A	FS-iA10B / FlySky systems use the AFHDS 2A 2.4GHz radio protocol for the control link.	The FS-iA10B receiver specs list the 2.4GHz system as AFHDS 2A [3].
Expected Operating range: > 50m	The system needs to have wireless connection that works for at least 50m, per our engineering requirements.	The FS-iA10B receiver specs list the range of the receiver as 500 - 1500m (in the air) [3].
Data carried: Battery voltage +- 1% error	The system needs to relay live readings of battery voltage to the user during flight, per our engineering requirements.	The FS-iA10B includes a SENS port with its IBUS protocol to be able to read sensor data (voltage) [3]. The FS-CVT01[4] is a voltage sensor designed to measure

		voltage and feed into the SENS port on the receiver. This combined system will allow the receiver to relay battery readings to the remote controller's display.
--	--	---

controller_reciever_rf: Input

Protocol: AFHDS 2A	FS-i6X / FlySky systems use the AFHDS 2A 2.4GHz radio protocol for the control link.	The FS-i6 remote controller specs list the 2.4GHz system as AFHDS 2A [1].
Expected operating range > 50m	The system needs to have wireless connection that works for at least 50m, per our engineering requirements.	Range is specified with the FS-IA10B receiver rather than in the FS-i6x manual. The FS-IA10B specs list a range of 500-1500m (in the air)[3].
Channel count supported: 4	A minimum of 4 channels is required for basic VTOL flight control. 4 channels was chosen to show proof of concept. The FS-6X and ia10b system can utilize up to 10 channels.	The FS-i6X remote controller specs list 6-10 channels [2]

3.2.5. Verification Process

controller_reciever_rf: Input:

- Protocol: AFHDS 2A
 1. Per manufacturer specs, the receiver uses AFHDS 2A rf protocol.
 2. Confirm that the receiver and remote controller are binded (linked to each other)
 3. Turn remote controller on
 4. Have receiver on (connected)
 5. Turn receiver off
 6. Wait 10 seconds and observe remote controller start to beep (lost connection to receiver)
 7. Turn receiver back on
 8. Wait 10 seconds and observe the remote controller do 2 beeps to signify connection and then no more beeping (regained connection to receiver).
 9. Confirm Solid led on receiver.
 10. This confirms they are binded.
- Range > 50m (Confirmed via video)

1. Keep receiver powered and connected to ESP32
 2. Walk 50m away with remote controller (in line of sight)
 3. Watch ESP32 serial output for channel values
 4. Confirm channel values continue updating and changing at 50m
- Channel count (4):
 1. Confirm that the receiver can receive at least 4 channels.
 2. On ESP32, print channels 4 from iBUS
 3. Move sticks (mapped to channels 1-4).
 4. Confirm serial output is accurate based on the serial monitor.
 5. Accuracy is based on if you move a joystick up or down (say the right joystick up corresponds to channel 2) then you will observe this on the serial monitor on channel 2.

reciever_controller_rf: Output

- Protocol: AFHDS 2A
 1. Per manufacturer specs, the remote controller uses AFHDS 2A rf protocol.
 2. Confirm that the receiver and remote controller are binded (linked to each other)
 3. Turn remote controller on
 4. Have receiver on (connected)
 5. Turn receiver off
 6. Wait 10 seconds and observe remote controller start to beep (lost connection to receiver)
 7. Turn receiver back on
 8. Wait 10 seconds and observe the remote controller do 2 beeps to signify connection and then no more beeping (regained connection to receiver).
 9. Confirm Solid led on receiver.
 10. This confirms they are binded.
- Range > 50m (Confirmed via video)
 1. Keep receiver powered and connected to ESP32
 2. Walk 50m away with remote controller (in line of sight)
 3. Watch ESP32 serial output for channel values
 4. Confirm channel values continue updating and changing at 50m
- Data carried: Battery voltage
 1. Connect FS-CVT01 (voltage reader) to DC power supply.
 2. Send a voltage (10V)
 3. Confirm the DC power supply voltage is the same voltage being displayed on the remote controller display +- 1% error.
 4. Wait 10 seconds and confirm voltage is still accurately displayed.

reciever_fc_dsig: Output

- Protocol: UART iBUS
 1. On ESP32, print channel inputs from the receiver.

2. Confirm channel values change when you move joy stick controls on remote controllers.
- Datarate: 115200 baud
 1. Set ESP32 UART baudrate to 115200.
 2. Baudrate is bits per second. So period per bit = $1 / 115200 = 8.68\mu\text{s}$
 3. Using an oscilloscope, measure the period between 1 bit.
 4. Confirm the period per bit is roughly $8.68\mu\text{s}$
 - Logic level
 1. Using an oscilloscope, measure the iBUS signal at the ESP32 Rx pin (after the voltage divider / level shifter)
 2. Confirm logic high voltage is approximately 3.3V (between 3.1 and 3.45V.)

3.2.6. References

- [1] "FS-i6X," Flysky, <https://www.flysky-cn.com/fsi6x> (accessed Jan. 22, 2026).
- [2] FS-i6X user manual, <https://static1.squarespace.com/static/5bc852d6b9144934c40d499c/t/5d6f66305487dc0001e34939/1567581800555/FS-i6X+User+manual+20160817.pdf> (accessed Jan. 23, 2026).
- [3] "IA10B-specifications," Flysky, <https://www.flysky-cn.com/ia10b-canshu> (accessed Jan. 22, 2026).
- [4] "CVT01-specifications," Flysky, <https://www.flysky-cn.com/cvt01-canshu> (accessed Jan. 22, 2026).
- [5] Bmellink, "Bmellink/IBusBM: Arduino Library for RC ibus protocol - servo (receive) and sensors/telemetry (send) using hardware UART," GitHub, <https://github.com/bmellink/IBusBM?tab=readme-ov-file> (accessed Jan. 22, 2026).
- [6] Espressif Systems, "ESP32-S3-WROOM-1 & WROOM-1U Datasheet(v1.5)", Doc. No. ESP32-S3-WROOM-1_WROOM-1U_Datasheet_EN, 4.0 years (≈ 2021) old. [Online]. Available: https://www.espressif.com/sites/default/files/documentation/esp32-s3-wroom-1_wroom-1u_datasheet_en.pdf
- [7] "Single wire flysky ibus telemetry: Betaflight," Betaflight News RSS, <https://www.betaflight.com/docs/wiki/guides/current/IBus-telemetry> (accessed Jan. 22, 2026).

3.3.Code block

This block contains the code for the Solar-Glide VTOL Drone, created by Carlos Alvarado Lopez. The program includes microcontroller coding using C++ and the implementation of sensor programming for data collection. Contents include code interfaces and properties, description and design of the code, general validation, and verification process.

3.3.1.Description

The Code block contains the embedded software implementation responsible for high-level system coordination, sensor processing, and communication for the Solar-Glide VTOL Drone. Software is implemented using two different microcontrollers: a custom ESP32 board used for the Flight Control block and an STM32F030F4P6 microcontroller used for the Prop Control block [1]. The STM32F030F4P6 microcontroller receives a PWM signal from the Flight Control block, which it uses to drive the MOSFETs in the ESCs. The custom ESP32 board is used for real-time processing, sensor integration, signal interpretation, and management of sensor data. For the ESP32, the ESP32-S3-WROOM-1U-N16R8 was chosen and implemented to handle these functions [2].

C++ code was chosen and implemented for the Solar-Glide VTOL Drone electronics. The program initializes communication, sensors, motors, and power monitoring, then continuously reads flight-data and outputs the data to the user. Data collection includes altitude and temperature data from a BMP581 barometer, drone orientation data from a BNO085 IMU (inertial measurement unit), and battery voltage through an analog input [3] [4]. Another function is the ability to read pilot commands from an RC receiver and convert them to control values. Motor outputs are handled through the motor interface, converting ESC PWM signals to throttle commands. During system flight and operation, the system prints out diagnostics for data collection and debugging. This allows for proper testing and verification of our sensors to ensure diagnostics are complete.

3.3.2. Design

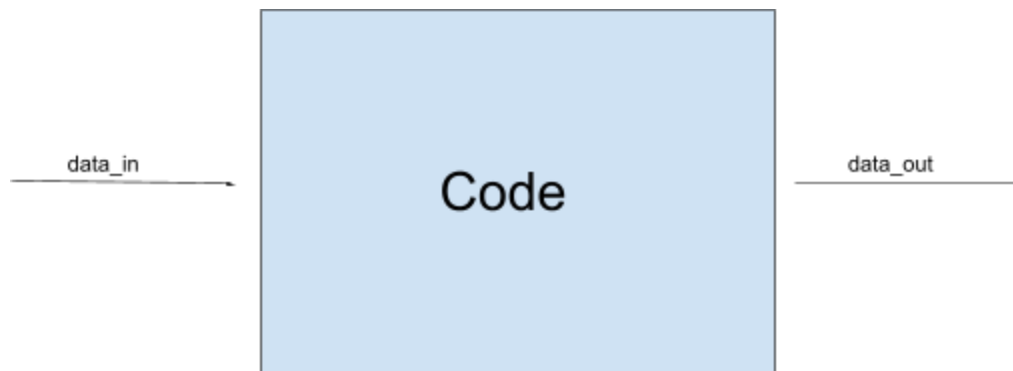


Fig. 1: Black box diagram of the flight controller and its interfaces

System startup:

- Initialize STM32 peripherals
- Initialize ESP32 communication interfaces
- Establish inter-processor communication

Main loop (STM32):

- Read sensor data
- Update system state
- Execute control logic
- Perform safety checks
- Send status data to ESP32

Main loop (ESP32):

- Receive telemetry from STM32
- Transmit telemetry wirelessly
- Receive commands
- Forward validated commands to STM32

Fig. 2: Pseudocode flow of Solar_Glide VTOL Drone

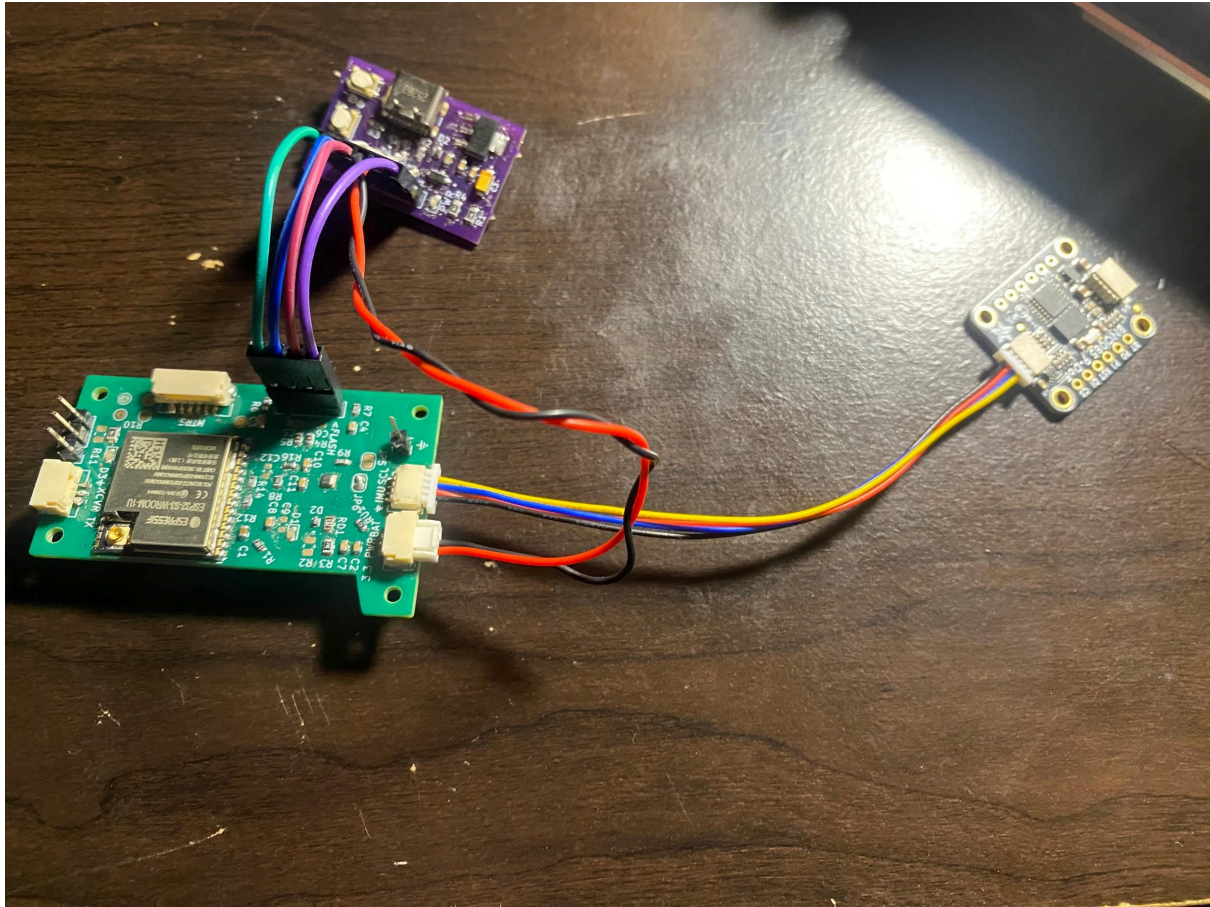


Fig.3: ESP32 and Flight Controller hooked up for testing via flasher PCB

For the design process for the code, it was decided early during the research phase to implement two microcontrollers, one for the sensor management and one for motor control. Since these blocks were created by my other team members, my role was to program the ESP32 and STM32 microcontrollers. I first began by programming the ESP32 board to ensure our chosen sensors were working properly. Afterwards, I collaborated with Noah Amack to determine the programming for the STM32 microcontroller and the motors.

3.3.3. General Validation

The Code block design meets the needs of the Solar-Glide VTOL Drone by providing a structured way to collect sensor data, process pilot commands, generate motor outputs, and report system status. The separation of data into `data_in` and `data_out` ensures that all information entering the control logic is validated before being used and that all outputs are generated in a controlled and predictable manner.

The use of normalized variables for key inputs and outputs ensures that the software remains independent of specific hardware timing. Sensor values such as altitude and orientation are converted into standard SI units, which makes them easy to verify against expected ranges and reference measurements. Similarly, RC inputs and motor commands are normalized, allowing consistent scaling, clamping, and safety checks regardless of the underlying communication protocol or PWM implementation.

The distributed processing approach using an STM32 microcontroller and a custom ESP32 board further supports system reliability [1] [2]. Motor command generation is handled locally on the microcontroller, while communication, sensor management, and telemetry tasks are the focus of the ESP32. This prevents communication overhead from interfering with real-time control tasks and ensures that system timing requirements are met.

Encapsulation of hardware-specific behavior through dedicated functions and classes allows safety constraints to be enforced consistently across the system. For example, motor commands are clamped before conversion to PWM outputs, and battery and RC validity checks prevent unsafe operation under fault conditions. This design reduces the likelihood of unintended behavior and ensures that failures result in predictable and safe outcomes.

Finally, the Code block design supports straightforward validation and verification. Each major input and output can be independently observed through serial logging, telemetry, and electrical measurements, making it possible to confirm correct behavior without requiring full flight testing.

3.3.4.Interface Validation

Interface property *Be sure to use the naming convention "from_to_type" here.	Why is this interface property this value?	How do you know your design details will meet or exceed this property? Cite your sources in IEEE.
--	--	---

Interface name: data_in

Altitude_m (0-10 feet)	Altitude is required for verifying vertical state information. Meters are the standard SI unit and align with barometer sensor output models. This is especially important for verifying our ER of altitude.	Altitude is obtained from the BMP581 barometric pressure sensor and converted to altitude using the pressure-to-altitude method. The code establishes a relative altitude by zeroing at startup, which reduces fixed offset error for validation. Verification is performed by logging altitude over serial and confirming the expected direction and magnitude of change when the sensor is moved between two known height references or a measured vertical displacement [3]
bmp.temperature (F)	The BMP581 measures the temperature of the environment. The data collection is in Celsius and is converted to Fahrenheit via mathematical calculations in the program.	Temperature collection is included in the barometer, it is verified by using an external temperature sensor to ensure readings are accurate.
Bool readBNO085 (variables: rc_roll, rc_pitch, rc_yaw) (-90-90 degrees)	The BNO085 IMU is responsible for detecting and measuring orientations in the X, Y, and Z planes. These are assigned to the roll, pitch, and yaw variables and print out the degrees in the serial monitor.	The IMU is moved to simulate the degree shifts of an operational drone. The serial monitor prints out these values for debugging purposes, as well as to limit the orientation of the drone to prevent imbalances during flight.

--	--	--

Interface name: data_out

Code_motors_cmd (variables: motor_cmd_qm1 ...qm4, motor_cmd_gm)	Normalized commands make it easy to clamp outputs, apply mixing, and keep motor logic hardware-independent. This also simplifies verification and tuning.	Motor commands are generated as normalized values and clamped in software before output mapping. Verification is performed by running a dedicated bench test mode that assigns known command values to each motor variable and confirming printed command values match the assigned targets.
code telemetry_frame (variables: out.temperature F)	Telemetry is necessary for debugging, validation, and evidence collection (tables, plots, flight logs). Values mirror the data_in state to make verification straightforward.	Telemetry values are derived directly from validated data_in variables to avoid inconsistency and to make debugging straightforward. Verification is performed by checking that telemetry outputs match internal state values within the same loop iteration.
Function: float PWM_PW_To_DutyPercent(uint16_t us);	ESCs are controlled using servo-style PWM signals defined by pulse width. The ESP32 PWM peripheral requires duty-cycle values rather than absolute pulse widths. This function converts a pulse-width command into a duty-cycle percentage so that motor commands can be applied using the PWM hardware while preserving standard ESC timing behavior.	The function is verified by confirming it produces the correct duty-cycle percentage for known pulse-width inputs based on the configured PWM period. This is verified by computing the expected duty% for test inputs, running the firmware so it uses those pulse widths, and measuring the resulting PWM waveform on the motor output pin with an oscilloscope/logic analyzer to confirm the high-time equals the target pulse width.

3.3.5.Verification Process

1. Connect the flight controller and ESP32 board according to the system wiring diagram.
2. Power the system via a USB connection to a computer to enable programming capabilities.
3. Build and upload code to the ESP32, and open the serial monitor. Ensure the correct baud rate of 11520.
4. Confirm successful startup by observing initialization messages in the serial monitor. Error messages will print for debugging assistance.
5. Verify altitude input by leaving the system stationary for at least 10 seconds and confirming that altitude readings remain stable with minimal drift.
6. Move BMP581 up and down to verify changes in the serial monitor.
7. Move the barometric sensor vertically between two known height references and confirm that the reported altitude changes in the correct direction and by a reasonable magnitude.
8. Verify IMU orientation inputs by slowly rotating the system about each axis and confirming that roll, pitch, and yaw values change smoothly without sudden jumps.
9. Observe outputs for the battery readings if connected to the system.
10. Enable a motor output test mode or manually assign known values to motor command variables.
11. Confirm that motor command variables remain within the range of 0 to 1 and match the expected test values.
12. Observe telemetry output over serial or wireless communication and confirm that telemetry values match internal system state values.
13. Record serial logs, measurement data, and screenshots or videos as evidence of correct block operation.

3.3.6. References

- [1] St,
<https://www.st.com/content/ccc/resource/technical/document/datasheet/a4/5d/0b/0e/87/c4/4d/71/DM00088500.pdf/files/DM00088500.pdf/jcr:content/translations/en.DM00088500.pdf> (accessed Nov. 2, 2025).
- [2] Espressif Systems, “ESP32-S3-WROOM-1 & WROOM-1U Datasheet (v1.5)”, Doc. No. ESP32-S3-WROOM-1_WROOM-1U_Datasheet_EN, 4.0 years (≈2021) old. [Online]. Available:
https://www.espressif.com/sites/default/files/documentation/esp32-s3-wroom-1_wroom-1u_datasheet_en.pdf
- [3] Bosch Sensortec GmbH, "BST-BMP581-DS004-13 Barometric Pressure Sensor – Datasheet", doc. no. BST-BMP581-DS004-13, Apr. 2025. [Online]. Available:
<https://www.bosch-sensortec.com/media/boschsensortec/downloads/datasheets/bst-bmp581-ds004.pdf>
- [4] CEVA Inc., “BNO080/085 Datasheet – 9-Axis SiP Sensor Hub Family”, Doc. No. 1000-3927 v1.17, 2023. [Online]. Available:
https://www.ceva-ip.com/wp-content/uploads/BNO080_085-Datasheet.pdf
- [5] GitHub, “Solar_Glide_VTOL/flightcontrol/testcode”
https://github.com/reefstarnes/Solar_Glide_VTOL/tree/main/Flightcontrol/testcode

3.4. Power Distribution Block

This is the Power Distribution Block, created by group member Carlos Alvarado Lopez. The documentation below contains the description of the block, design artifacts for the custom-made PCB, a general validation explanation, an interface validation table for all interface properties, and a verification process.

3.4.1. Description

The Power Distribution Block (PDB) is responsible for regulating and distributing voltage to all drone subsystems. The system is powered by a 6200 mAh 22.2V 6s 90 c Li-Po battery, but several components require different voltage levels for safe operation [1]. To achieve this, multiple buck converters are implemented on a custom-made PCB. These include an AP63203 buck converter for 3.3V and a TPS56A37 buck converter for 5V [2] [3]. 3.3V powers the drone's microcontrollers, and 5V powers the servos. To power the motors, the battery will connect to the ESCs in the Prop Control PCB to safely operate them.

The usage of power rails in the form of bus bars is implemented to power the ESCs. The battery is connected to the bars, which will evenly distribute the voltage to the 5 ESCs we are using, which are connected to the five motors we have. These bus bars are rated for a higher rating of current, as we designed this system for a worst-case scenario of 100% throttle for an extended period of time for all motors. Using this system, we can safely handle these current levels and have the PDB solely be responsible for powering the microcontrollers and servos.

3.4.2.Design

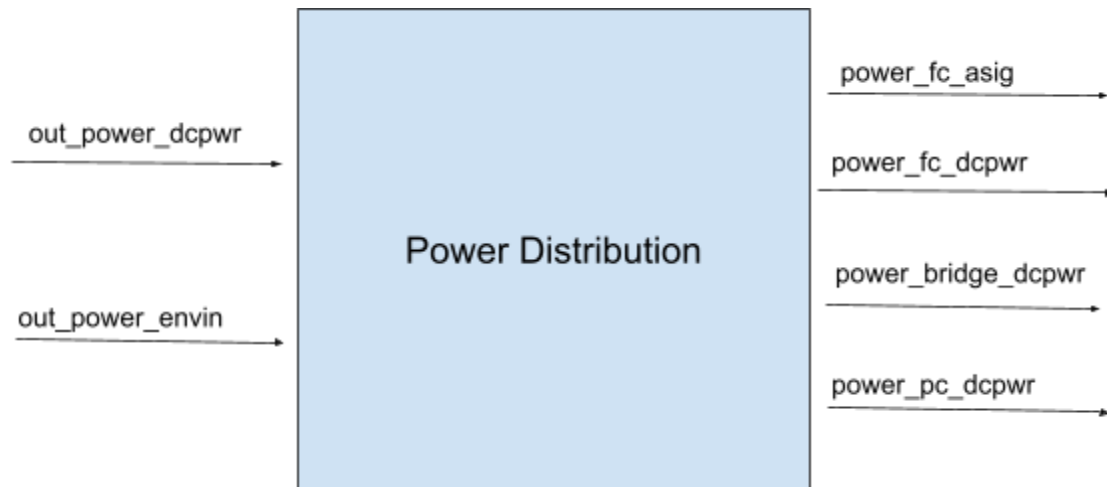


Fig. 1: Black box diagram of the Power Distribution block and its interfaces.

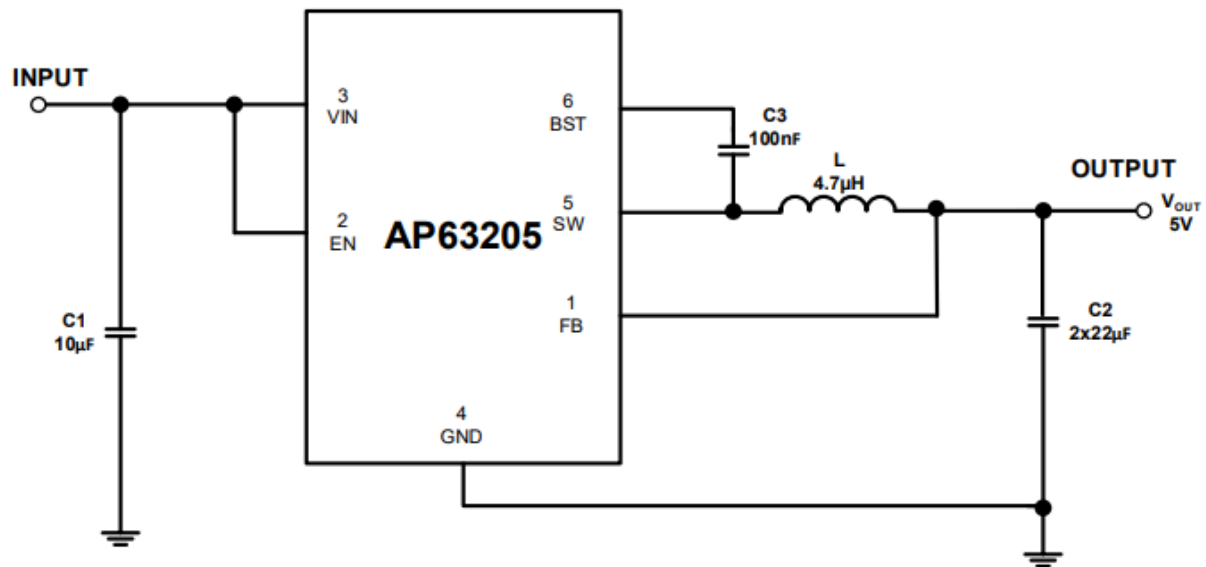


Fig. 2: Reference schematic for AP63203 synchronous buck converter [2].

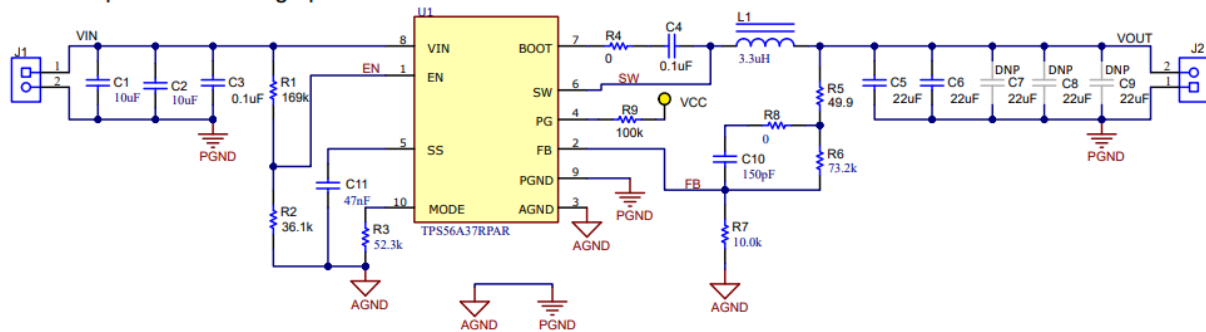


Fig. 3: Reference schematic for TPS56A37 synchronous buck converter [3].

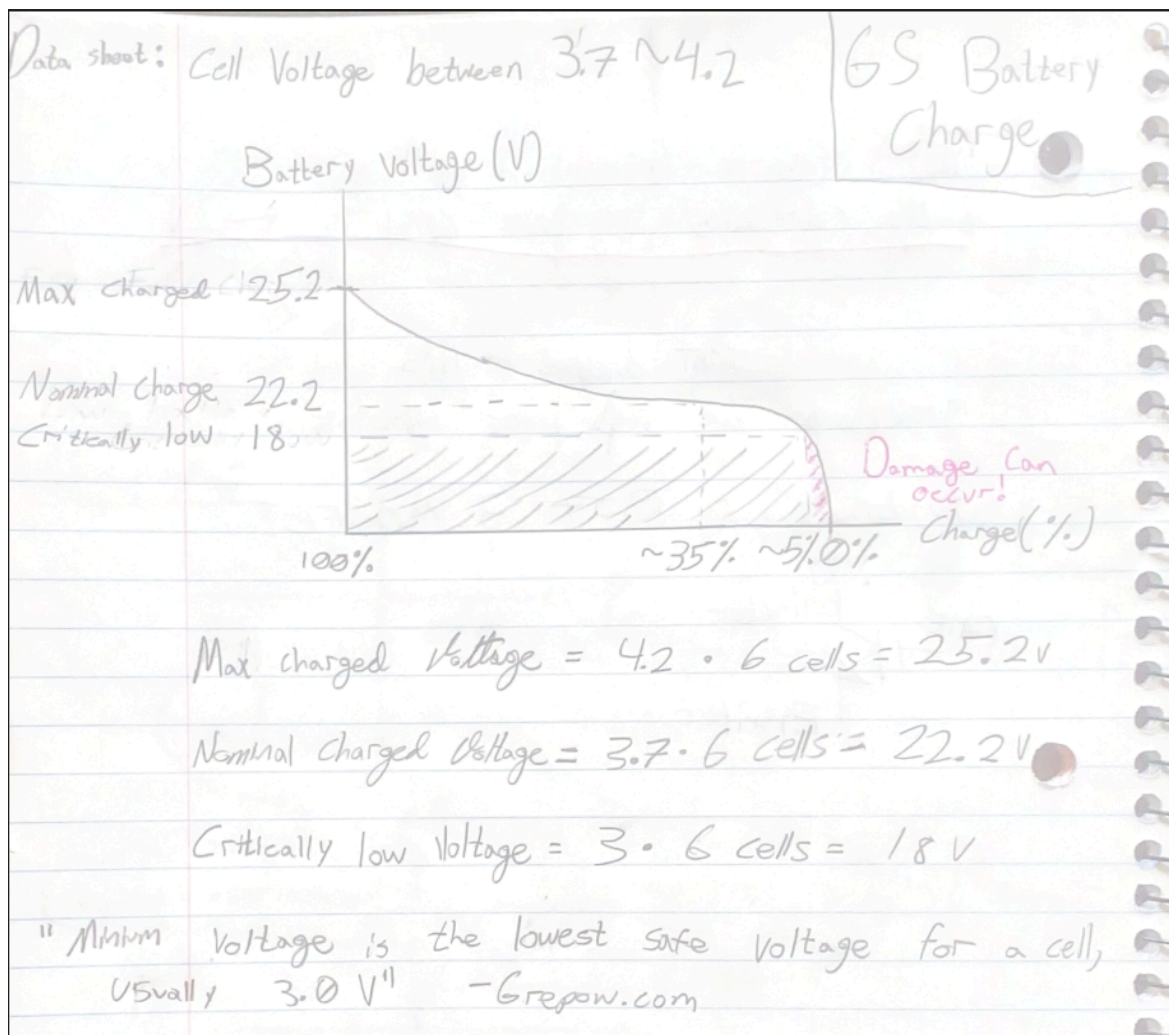


Fig. 4: Battery voltage calculations for out_power_dcpwr

3.4.3.General Validation

The design meets system requirements by providing power paths for all drone components. Component selections were based on verified electrical characteristics and datasheet specifications.

Our chosen motors, which include 4 1350KV 6s brushless motors and 1 900KV 6s brushless motor, are connected to the PDB through ESCs in the prop control block [4] [5]. These custom ESCs are rated for 6S operation (22.2V) and connect directly to the main battery bus.

For the flight control block, the ESP32 required an operating voltage range of 3-3.6V, and the prop control microcontroller required an operating voltage of 2.4-3.6V [6] [7]. To attain this voltage, the AP63203 buck converter was chosen due to its input voltage range being well within the range of the Li-Po battery and its consistent 3.3V output [2]. This voltage was well within the operating range of the microcontrollers, leading to its selection.

For the servos, the operating voltage is 5V. This led to the selection of the TPS56A37 buck converter, due to its output of 5V and input voltage being in the range of the battery [3].

3.4.4.Interface Validation

Interface property		
*Be sure to use the naming convention “from_to_type” here.	Why is this interface property this value?	How do you know your design details will meet or exceed this property? Cite your sources in IEEE.

Interface name: out_power_dcpwr

Vmin: 18V	Figure 4 in the Design section shows the calculations for the minimum voltage based on the number of cells and the cell voltage. This fits within the voltage input range of the buck converters.	Recommended operating voltages can be found in the datasheets under the recommended operations section [2, pg.4] [3, pg.4].
-----------	---	---

V _{max} : 25.2V	Figure 4 in the Design section shows the calculations for the maximum voltage based on the number of cells and the cell voltage. This fits within the voltage input range of the buck converters.	Recommended operating voltages can be found in the datasheets under the recommended operations section [2, pg.4] [3, pg.4]
I _{nominal} : 50 A	Adding the current values of intended throttle ranges for the motors sums to 50A.	These values can be found in the testing data table provided on the motor's website [4].
I _{peak} : 236.6A	Adding all the peak current values of the motors in the worst-case scenario of all motors at 100% throttle results in this peak current draw.	These values can be found in the testing data table provided on the motor's website [4].

Interface name: out_power_envin

env_power_voltage_max = 25 V	The maximum open-circuit voltage (V _{oc}) of a 12 V nominal solar panel is typically 21–23 V at STC. A 25 V ceiling includes margin for cold-temperature V _{oc} rise.	All input-stage components (buck converter, input capacitors, TVS diode) are rated ≥ 30 V absolute maximum. Capacitors selected at 35 V rating. Converter datasheet confirms safe operation within this range.
env_power_current_max = 2.0 A	Maximum short-circuit current (I _{sc}) for small UAV-grade solar panels in the 20–30 W range is typically ≤ 2.0 A. This defines trace width, fuse sizing, and connector rating.	Input fuse rated 3 A hold. Connector rated ≥ 5 A. Datasheets confirm all exceed 2.0 A requirement.
env_power_power_max = 30 W	A 12 V nominal panel with V _{mp} ≈ 18 V and I _{mp} ≈ 1.6–1.8 A produces ≈ 28–32 W. Designing for 30 W ensures the power stage can safely process full solar input.	The selected DC-DC converter/charger supports ≥ 40 W input power with thermal margin.

Interface name: power_fc_asig

Vrange: 0-25.2	This represents the voltage range output on the analog signal line. It depends on the charge level of the 6S LiPo battery, where each of the six cells can charge up to a maximum of 4.2 V [4]. $6 \text{ cells} \times 4.2 \text{ V} = 25.2 \text{ V}$	A voltage divider is utilized to linearly step down the asig signal by 2/17 (as seen in Figure x), reliably providing a safe voltage range for the ESP32 <u>under</u> our operating voltage [1, p. 29]. $V_{out} = \frac{2}{17} V_{in}$ $V_{out}(25.2) \approx 2.96 \text{ V}$ $V_{out}(0) = 0 \text{ V}$
Minimum Sampling Frequency: ≥ 1 Hz	This is the minimum rate at which the ADC samples battery voltage. Providing sufficient resolution to capture slow, gradual battery discharge behavior. This low frequency minimizes processing overhead.	A 1 Hz sampling frequency is well within the capabilities of the ESP32 [1, p.28].
Frequency: 0 Hz	The battery (or DC supply) provides a DC voltage that does not oscillate, so its fundamental frequency is effectively 0 Hz.	The ESP32-S3 ADC supports DC and low-frequency analog inputs, so reading a 0 Hz signal is fully within its capabilities [1, p.28].
Read Resolution: 0.75 V	This represents the voltage resolution of the battery measurement. Such variation is negligible in terms of motor performance and battery life.	The ESP32 GPIO4 pin uses a 12-bit ADC, providing 2^{12} possible digital values. For an input range of 0V to 25.2 V, the corresponding voltage step size is: $(25.2 - 0) \div (2^{12} - 1)$ $\sim 6.15 \text{ mV}$ This means the ADC can distinguish changes as small as 6.15 mV. For readability

		and practical monitoring, the value will be rounded to increments of 0.75 V.
--	--	--

Interface name: power_fc_dcpwr

Vmin: 3.10V	3.0V is the minimum amount of voltage that the ESP32 Microcontroller, BMP581 Barometer and BNO085 IMU can have input [1, p. 29][2, p.10][3, p.35]. A small safety margin of 0.1V is added to ensure proper operation.	The AP6320 buck converter's 3.3V output is above this minimum voltage range [14].
Vmax: 3.45V	This is the maximum amount of voltage that the ESP32 Microcontroller, BMP581 Barometer and BNO085 IMU can have input [1, p. 29][2, p.10][3, p.35]. A small safety margin of 0.15V is subtracted to ensure proper operation.	The AP6320 buck converter's 3.3V output is below this maximum voltage range [14].
Inominal: 50 mA	Normal operation draws a typical 355 mA, based on the active-mode specification where all ESP32 peripherals are enabled, including Bluetooth and Wi-Fi [1, p. 30]. The BMP581 and BNO085 add only 260 μ A and 100 μ A, which is negligible [2, p. 12][3, p. 51]. Bench testing shows a much lower draw of X mA, consistent with Bluetooth and Wi-Fi being disabled on the ESP.	The AP6320 buck converter's 2A output is above this nominal current range [14].

$I_{peak} = 100 \text{ mA}$	The ESP32's datasheet lists a maximum draw of 500 mA [1, p. 29]. Manual testing never exceeded 50 mA, and applying a x2 safety factor gives 100 mA.	The AP6320 buck converter's 2A output is above this nominal current range [14].
-----------------------------	---	---

Interface name: power_pc_dcpwr

$V_{min} = 2.4\text{V}$	This is the minimum operating voltage for the STM32F030F4P6 according to the datasheet[7, p.1].	The 6S-rated nominal 22.2 V power supply will be regulated to 3.3V through a buck converter [2], ensuring that the STM always operates above the minimum operating voltage.
$V_{max} = 3.6\text{V}$	This is the maximum operating voltage for the STM32F030F4P6 according to the datasheet[7, p.1].	The 6S-rated nominal 22.2 V power supply will be regulated to 3.3V through a buck converter[2], ensuring that the STM always operates below the maximum operating voltage.
$I_{nominal} = 24.56\text{mA}$	The nominal was calculated with the typical supply current (26mA) in run mode when flashed and supplied with 3.6V, according to the datasheet [7, p.47]. There will be 3.3V supplied: $I_{3.3nom} = I_{3.6nom} \times 3.3/3.6$ $= 26. \text{mA} \times 3.3/3.6 = 24.56\text{mA}$	The 6S-rated nominal 22.2 V power supply will be regulated to 3.3V through a buck converter[10], ensuring that the system operates at the peak current.
$I_{peak} = 27.68\text{mA}$	The nominal was calculated with the typical supply current (30.2 mA) in run mode when flashed and supplied with 3.6V, according to the datasheet [7, p.47]. There will be 3.3V supplied: $I_{3.3max} = I_{3.6max} \times 3.3/3.6$ $= 30.2\text{mA} \times 3.3/3.6 = 27.68\text{mA}$	The 6S-rated nominal 22.2 V power supply will be regulated to 3.3V through a buck converter[2], ensuring that the system operates at the peak current.

Interface name: power_bridge_dcpwr

$V_{min} = 18V$	This is the minimum safe voltage of a 6S Lipo battery corresponding to 3.0V per cell, which is the typical lower limit before cell degradation occurs [1].	The ESC's MOSFETS are rated for 0 to 60V according to the datasheet [8, pg.1], which will allow the system to remain fully functional at a minimum 18V input. The gate driver ensures reliable switching and conduction near this lower voltage.
$V_{max} = 25.2 V$	This is the maximum voltage of the fully charged 6S Lipo battery (4.2V per cell) according to the datasheet[1]	The ESC's MOSFETS are rated for a VDS (Drain to source voltage) of 60V [8, p.1], which provides over 2x a safety margin above the maximum battery voltage, ensuring safe operation.
$I_{nominal} = 10A$	Continuous current at 60% throttle [4] to lift the drone high enough to transition to forward flight.	Based on motor test data, at 60% throttle, the current draw is 10A, which is within the current handling capability of the ESC's MOSFETs (rated for 100A) [4, p.1]
$I_{peak} = 38.1 A$	Maximum current drawn by a single motor under 100% throttle [4].	Based on motor test data, at 100% throttle, the current draw is 38.1A, which is within the current handling capability of the ESC's MOSFETs (rated for 100A) [4, p.1].

3.4.5.Verification Process

Preparation:

1. Assemble the Power Distribution PCB with all regulators (AP63203, TPS56A37), connectors, and passive components soldered according to the schematic.
2. Connect the board to a programmable DC power supply set to 22.2 V nominal, representing the 6S Li-Po battery.
3. Attach a multimeter, oscilloscope, and electronic load for measurement and load testing.

4. Ensure all grounds between the instruments and the PCB are common.
5. Confirm the correct polarity of all power connections before powering the board.

Verify power_fc_dcpwr:

1. Connect the ESP32 microcontroller to the power distribution block or an external regulated DC power supply
2. For each test, keep power flowing for greater than 2 minutes to ensure stable voltage and current readings.
3. Set the power supply to the minimum voltage of 3V.
 - Use a multimeter to measure the VDD pin on the ESP32.
 - Confirm that the ESP32 powers on via the green flashing indicator LED and the voltage does not drop below 3V.
4. Set the power supply to the maximum voltage 3.6V.
 - Utilize a multimeter to measure the VDD pin on the ESP32.
 - Next confirm the ESP32 powers on via the green flashing indicator LED and the voltage does not drop below 3V.
5. Set the power supply to the nominal voltage (3.3V), representing the actual system operating condition.
 - Use a multimeter or read the power supply display to ensure the current drawn from the microcontroller is approximately its nominal current of 350 mA and that it stays below the peak current of 500 mA.

Verify power_fc_asig:

1. Connect a variable DC power supply to the battery input pins on the PCB.
2. Sweep the input voltage from 0V to 25.2 V, simulating the 6S LiPo batteries discharge range.
3. Use a multimeter to measure the voltage at the ADC pin and confirm it scales linearly to 2/17, while remaining below 3.3 V.
4. Verify that the ADC readings in the serial monitor can consistently update with 5 readings per second (5 Hz sampling rate).

5. Use an oscilloscope to ensure signal ripple remains below 100 mV p-p during measurement.

Verify power_pc_dcpwr:

1. Connect the STM32 microcontroller to the power distribution block or an external regulated DC power supply
2. For each test, keep power flowing for 1-2 minutes to ensure stable voltage and current readings.
3. Set the power supply to the minimum voltage (2.4V).
 - Use a multimeter to measure the VDD pin on the STM32.
 - Confirm that the STM32 powers on and the voltage does not drop below 2.4V.
4. Set the power supply to the maximum voltage (3.6V)
 - Use a multimeter to measure the VDD pin on the STM32.
 - Confirm that the STM32 powers on and the voltage does not exceed 3.6V.
5. Set the power supply to the nominal voltage (3.3V), representing the actual system operating condition.
 - Use a multimeter or read the power supply display to ensure the current drawn from the microcontroller is approximately its nominal current of 24.56mA and that it stays below the peak current of 27.68mA.

Verify power_bridge_dcpwr

1. Connect the ESC (specifically the drain of the 3 phase bridge mosfets) to the power distribution block or an external regulated DC power supply
1. For each test, keep power flowing for 1-2 minutes to ensure stable voltage and current readings.
2. Set the power supply to the minimum voltage (18V).
 - Use a multimeter to measure the drains of the mosfets to ensure that the voltage to the 3 phase full bridge remains above 18V.
 - Confirm that the gate drivers and MOSFETs switch currently at this voltage.
3. Set the power supply to the maximum voltage (25.2V)

- Use a multimeter to measure the drains of the mosfets to ensure that the voltage to the 3 phase full bridge remains below 25.2V
 - Confirm that the gate drivers and MOSFETs switch currently at this voltage.
4. Set the power supply to the nominal voltage (22.2V), representing the actual system operating condition.
- Apply a 60% throttle signal and observe the current draw.
 - Verify that the nominal current is approximately 10A.
 - Apply a 100% throttle signal and observe the current draw.
 - Verify that the peak current does not exceed 38.1A

3.4.6.References

[1] "CNHL racing series 6200mAh 22.2V 6s 90c lipo battery with EC5 plug," ChinaHobbyLine, <https://chinahobbyline.com/collections/cnhl-voltage-22-2v-6s-lipo-batteries/products/cnhl-racing-series-6200mah-22-2v-6s-90c-lipo-battery-with-ec5-plug> (accessed Oct. 26, 2025).

[2] AP63200/AP63201/AP63203/AP63205 document number: DS41326 rev. 3 - 2 1 of 18, <https://www.diodes.com/assets/Datasheets/AP63200-AP63201-AP63203-AP63205.pdf> (accessed Nov. 3, 2025).

[3] Ti, https://www.ti.com/lit/ds/symlink/tps56a37.pdf?ts=1754592259838&ref_url=https%253A%252F%252Fwww.ti.com%252Fproduct%252FTPS56A37%252Fpart-details%252FTPS56A37RPAR (accessed Nov. 3, 2025).

[4] "Deepspace Red Line 2807 brushless motor 1350KV 6s for FPV 7-8inch freestyle long range drones-1350KV," NewBeeDrone, <https://newbeedrone.com/products/deepspace-red-line-2807-brushless-motor-1350kv-6s-for-fpv-7-8inch-freestyle-long-range-drones-1350kv> (accessed Oct. 26, 2025).

[5] Banggood.com, "GEPRC EM3115 3115 900KV 6s brushless motor for 8 9 10 Inch long range FPV RC racing drone," [www.banggood.com](https://www.banggood.com/GEPRC-EM3115-3115-900KV-6S-Brushless-Motor-for-8-9-10-Inch-L), <https://www.banggood.com/GEPRC-EM3115-3115-900KV-6S-Brushless-Motor-for-8-9-10-Inch-L>

ong-Range-FPV-RC-Racing-Drone-p-2003873.html?gmcCountry=US¤cy=USD&cur_warehouse=CN&createTmp=1&ID=629197049289 (accessed Oct. 26, 2025).

[6] Espressif Systems, "ESP32-S3-WROOM-1 & WROOM-1U Datasheet (v1.5)", Doc. No. ESP32-S3-WROOM-1_WROOM-1U_Datasheet_EN, 4.0 years ($\approx$ 2021) old. [Online]. Available: https://www.espressif.com/sites/default/files/documentation/esp32-s3-wroom-1_wroom-1u_datasheet_en.pdf

[7] St, <https://www.st.com/content/ccc/resource/technical/document/datasheet/a4/5d/0b/0e/87/c4/4d/71/DM00088500.pdf/files/DM00088500.pdf/jcr:content/translations/en.DM00088500.pdf> (accessed Nov. 2, 2025).

[8] CSD18533Q5A Texas Instruments | Discrete Semiconductor products | DigiKey, <https://www.digikey.com/en/products/detail/texas-instruments/CSD18533Q5A/3674878> (accessed Nov. 2, 2025).

[9] Bosch Sensortec GmbH, "BST-BMP581-DS004-13 Barometric Pressure Sensor – Datasheet", doc. no. BST-BMP581-DS004-13, Apr. 2025. [Online]. Available: <https://www.bosch-sensortec.com/media/boschsensortec/downloads/datasheets/bst-bmp581-ds004.pdf>

[10] CEVA Inc., "BNO080/085 Datasheet – 9-Axis SiP Sensor Hub Family", Doc. No. 1000-3927 v1.17, 2023. [Online]. Available: https://www.ceva-ip.com/wp-content/uploads/BNO080_085-Datasheet.pdf

[11] Lipo battery voltage, discharge rate and cycle life | Grepow, <https://www.grepow.com/blog/basis-of-lipo-battery-specifications.html> (accessed Nov. 2, 2025).

3.5. Flight Control

The flight controller block is responsible for managing all real time processing, sensor integration, and actuator control required for stable aircraft operation. It receives user input from the FlySky FS-iA6b transceiver, processes feedback from onboard sensors including the BNO085 IMU, BMP681 barometer, and battery voltage to generate PWM signals to drive the ESCs and motors. The heart of this block is the ESP32-S3-WROOM-1U-N16R8, which handles this flight logic. This block ensures synchronized communication between all onboard systems. The block owner is **Reef Starnes**, who is responsible for its documentation, design and validation.

3.5.1. Description

The Flight Control block serves as the CPU for the aircraft, managing sensor data, signal interpretation, and control output sent to the ESCs. It interprets user commands received digitally through the FlySky FS-iA6B receiver via the UART-based IBUS protocol and integrates measurements from onboard sensors such as the BMP581 barometer and BNO085 IMU. These inputs are processed by the ESP32-S3-WROOM-1U-N16R8 microcontroller to compute stable flight responses and generate PWM signals that drive the ESCs and motors.

In addition to communication and control, the block handles battery voltage monitoring through an analog input circuit for emergency automatic landing when battery voltages get to low, a common safety feature found in most drones. It incorporates essential support components such as TV diodes for ESD protection and filtering capacitors (seen in Figures 2-7). Additionally, an external flashing device was created to offload some of the weight required for USB flashing, more details can be found in Figure 8 and its associated description. Overall, The Flight Control Block coordinates data flow between sensors, actuators, and the transceiver, forming the core of manual flight operation assisted by smart PID control found in the chip's created software.

3.5.2. Design



Fig. 1: Black box diagram of the flight controller and its interfaces.

To power the flight controller, 3.3 Volts are supplied through a connector to the flight controller.

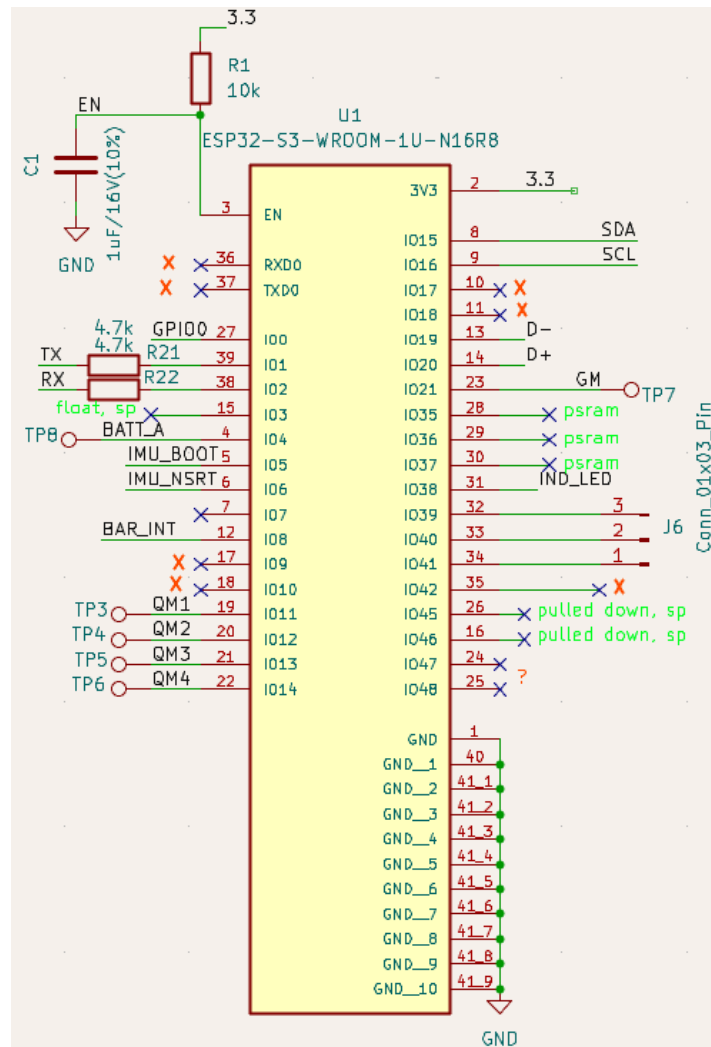


Fig. 2: Schematic diagram of the ESP32-S3-WROOM-1U-N16R8 and its connections. The positive and negative data lines (D+ and D- respectively) are used for flashing via the OGB. Figures 3-7 are apart of this same schematic, displayed separately to be viewed clearly, and modulated for clarity.

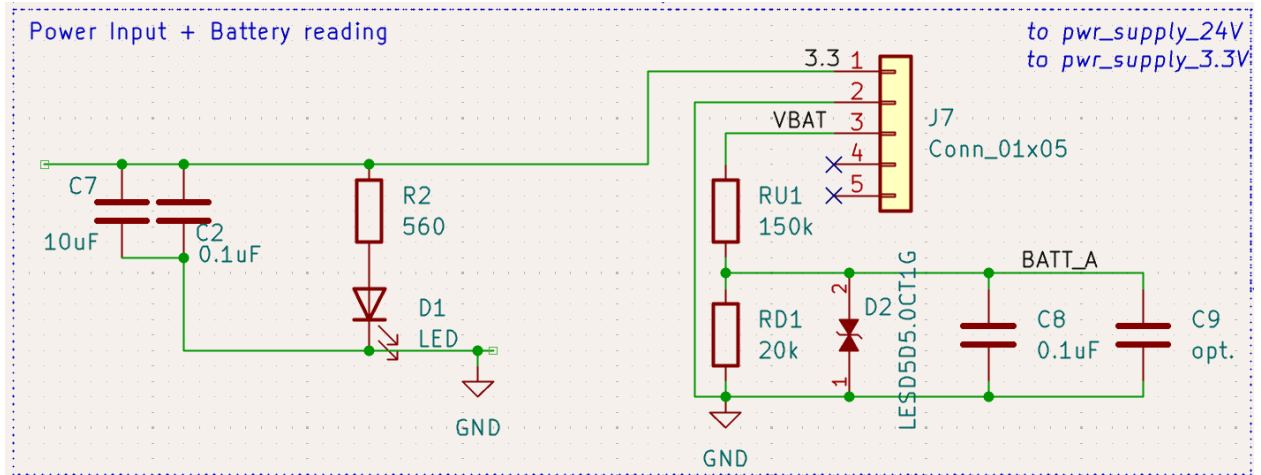


Fig. 3: On the right is a voltage divider connected to GPIO7 (BATT_A), which reads analog data to compute the battery voltage corresponding to the battery's charge level. More information can be found in `power_fc_asig:peaktopeak`. A 0.1 μF capacitor, as advised in the ESP32 datasheet for more reliable ADC conversion [5, p.17], was included. A TV diode was also added to ground to provide ESD protection from the battery when it is manually swapped by the user for charging. Fig. 4: On the left is a circuit for power input 3.3V with an LED indicator. Decoupling capacitors are added for smooth DC input. The connectors are jumped to the Power Distribution Block PCB.

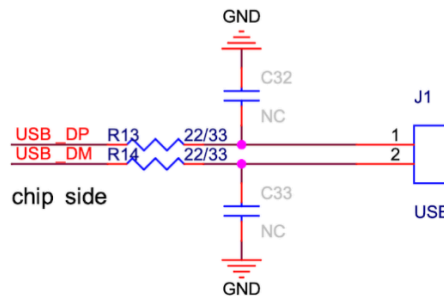


Fig. 4: Reference setup for flashing via positive and negative data lines. This circuit is required when using OTG USB flashing by the ESP32 Hardware Design Guidelines Document [5, p.18], and fully implemented in the figure below.

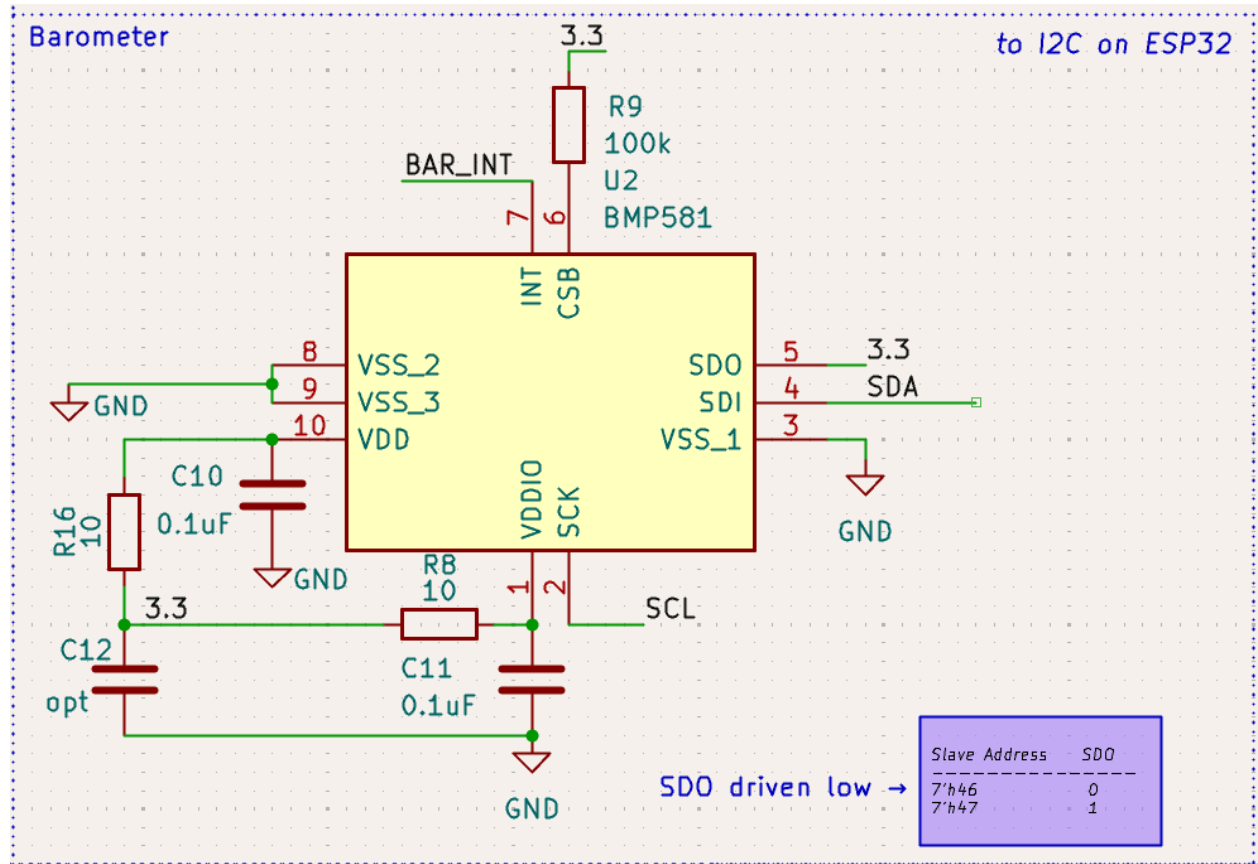


Fig. 6: Barometer implementation in which data is read via I²C by the ESP32. The CSB pin is pulled high through a 10 kΩ resistor, while VSS_2 and VSS_3 are connected to ground. Both the sensor supply voltage and logic voltage require 100 nF capacitors placed near their terminals and tied to 3.3 V. All according to the BMP581 data sheet [2, p.49].

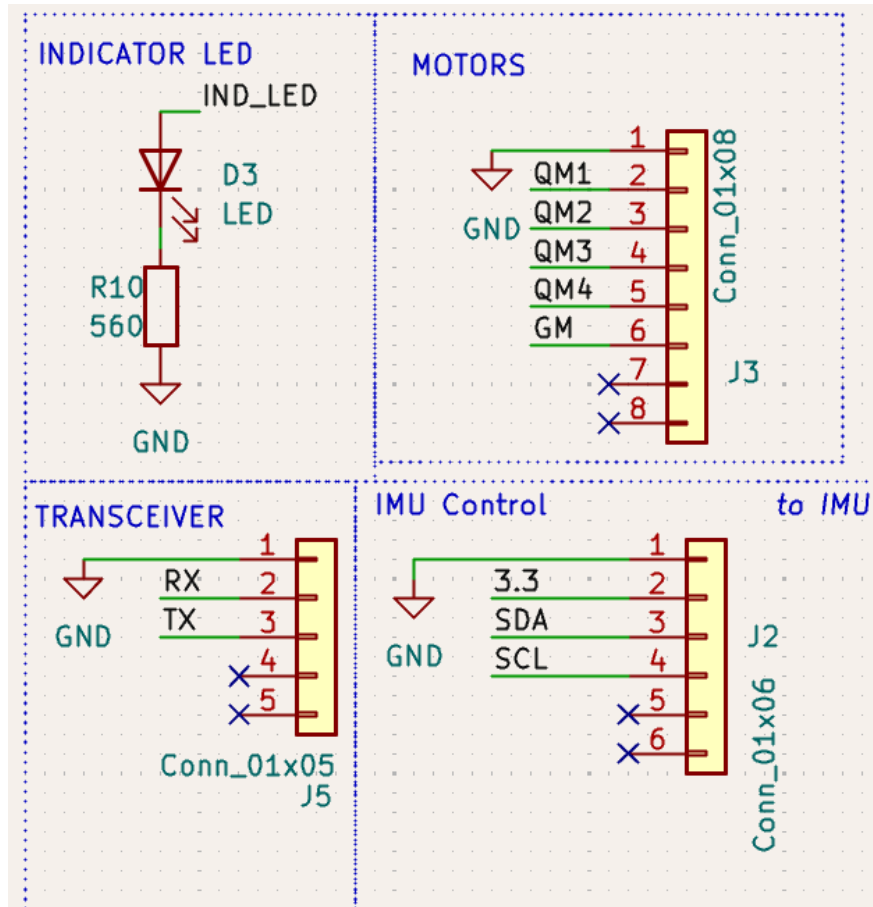


Fig. 7: Led indicator for debugging, motor PWM connector, IMU connector, transceiver and connector.

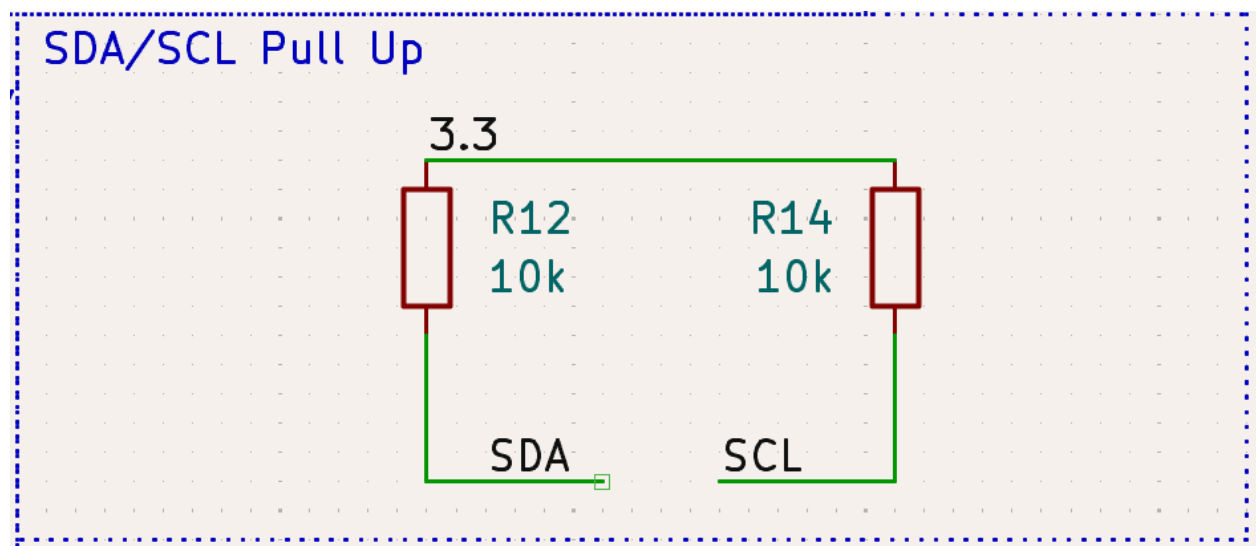


Fig. 8: Pull-up resistors are necessary for I2C connections [16]

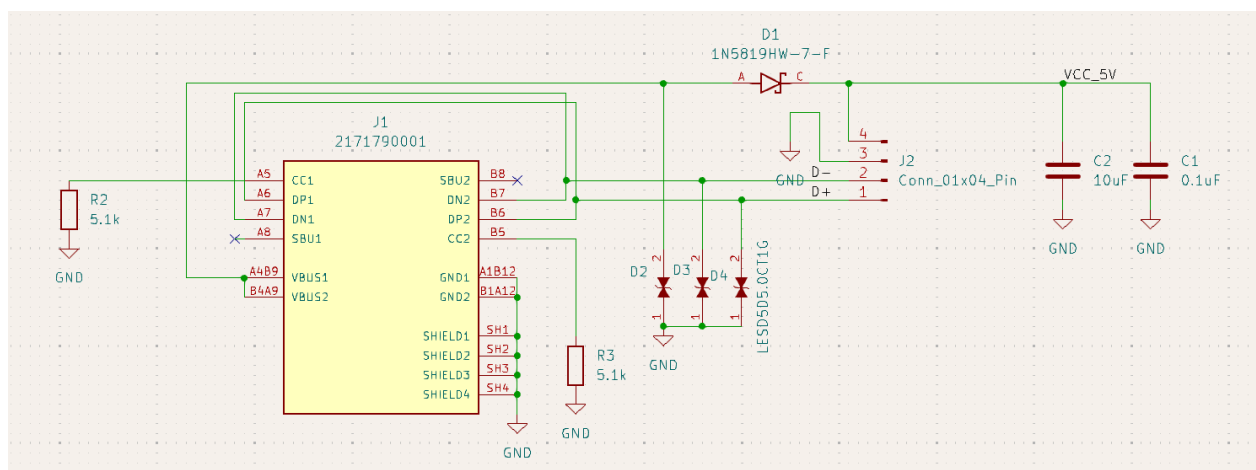
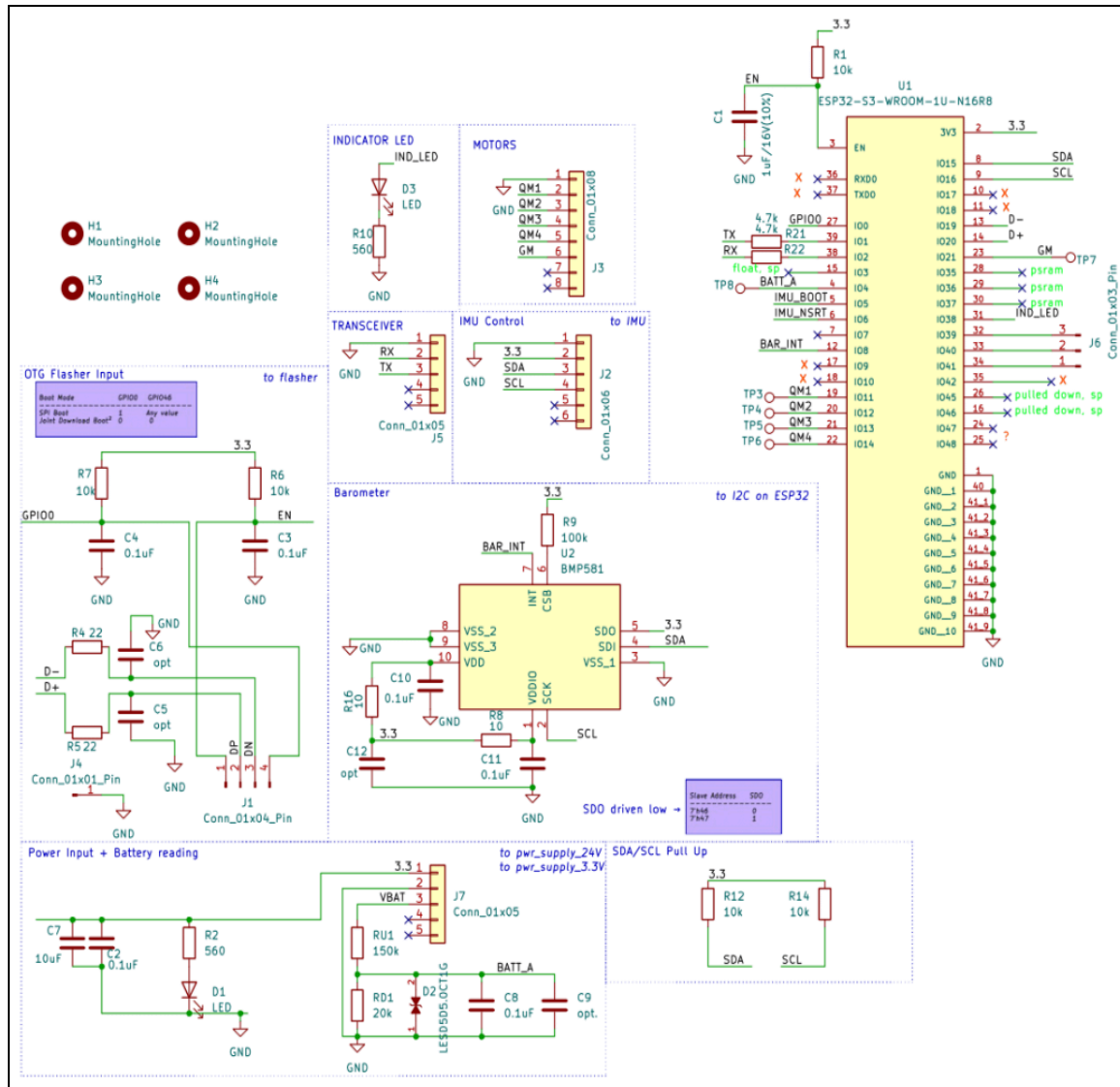


Fig. 10: OTG USB external flasher. This is a separate circuit from the others, on its own PCB, designed to offload some weight to make the aircraft lighter. It is a USB-C adaptation of a development board schematic portion to flash via D+, D-, and GND over 3 jumper cables externally [9].

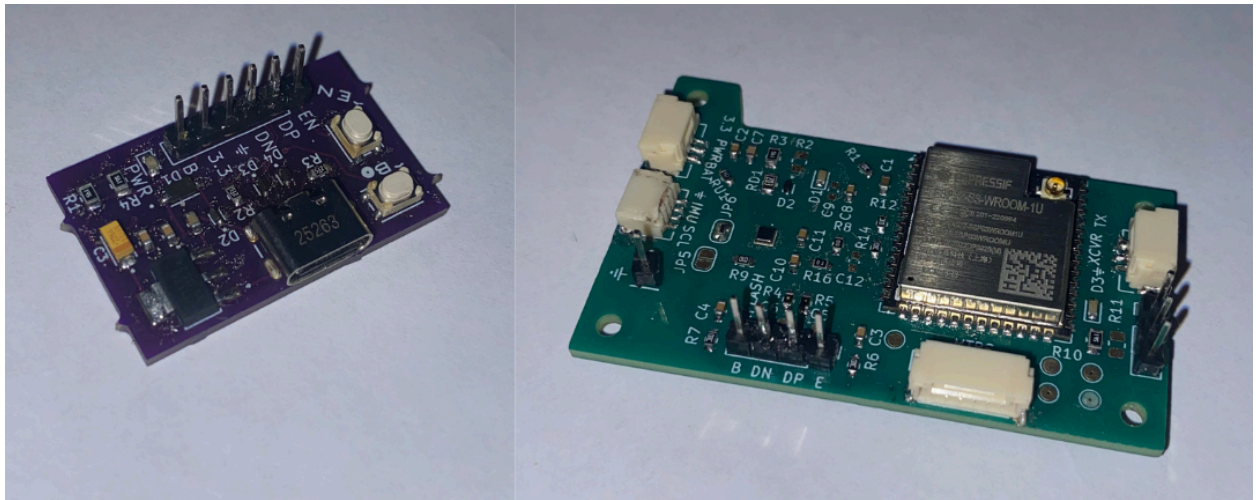


Fig. 11: Fully assembled Flasher OTG PCB V1 (left) and Flight Controller V1 PCB (right)

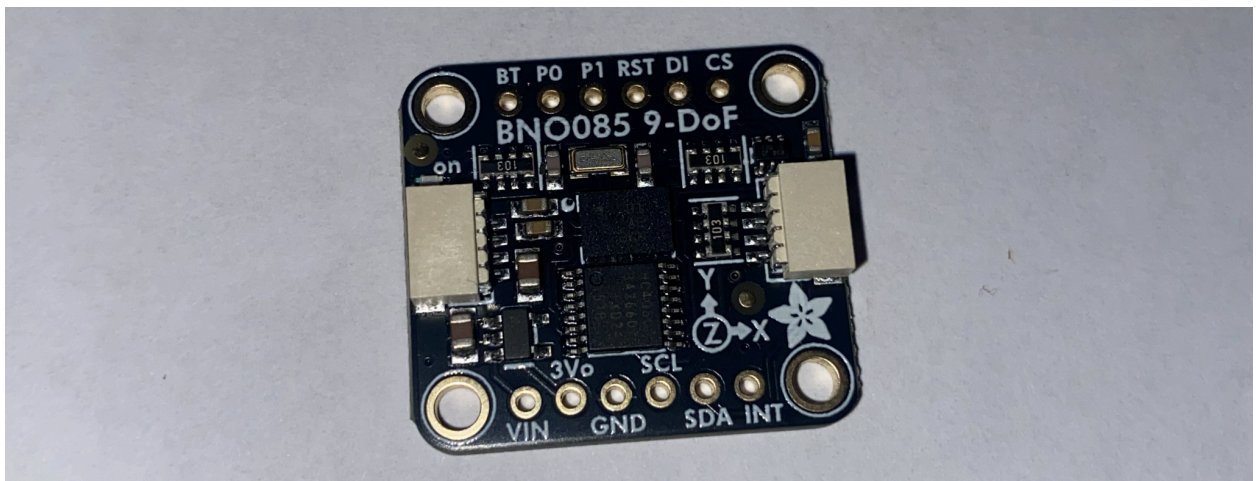


Fig. 12: Adafruit BNO085 IMU breakout board. Future versions will directly integrate the BNO085 onboard the Flight Controller PCB. For now a cable to communicate via I2C will suffice.

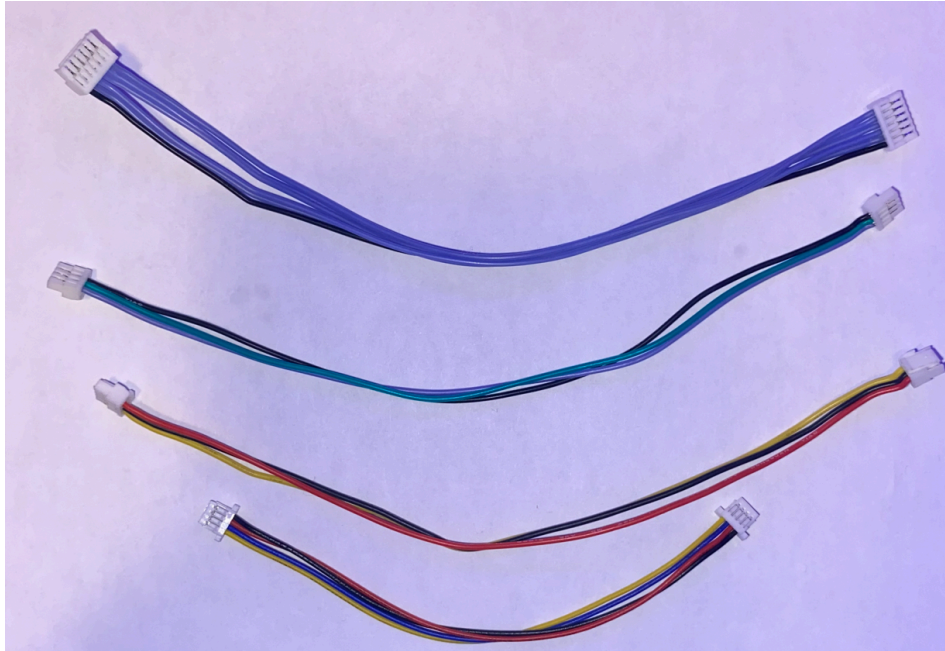


Fig. 13: Proper flight-controller cabling. From top to bottom: the propeller-control interface cable (E), the communication interface cable (A), the power cable (B), and the IMU breakout-board cable (Δ).

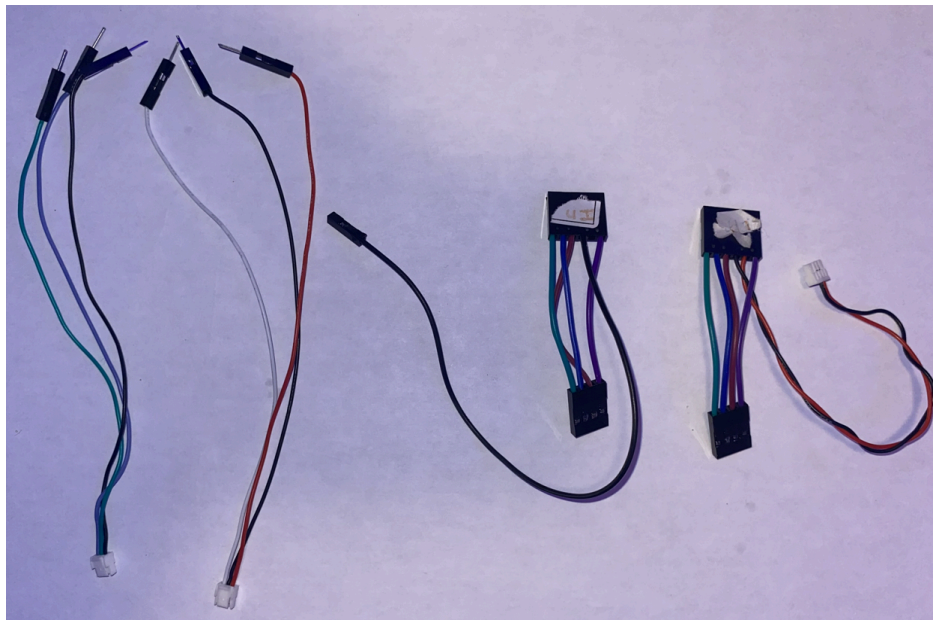


Fig. 14: Alternative flight-controller cabling. From left to right: the communication interface cable with Dupont male ends (α), the power cable with Dupont male ends (β), the Flasher OTG cable (λ) and the Flasher OTG cable with an added power connection (π).

3.5.3. General Validation

The block is powered from a 3.3 V rail, which lies precisely inside every connected device's allowed range (ESP32-S3, BMP581, BNO085). This is because the selected AP6320 buck converter provides ample current headroom to avoid brownouts during current spikes, along with capacitors near each device's input voltage on the PCB. The local decoupling and TVS ESD protection at these sensitive nodes combat noise under vibration and sudden current spikes.

Battery voltage is measured and converted to battery charge, to be relayed back to the user via the FS-iA6b. This is accomplished by a 2/17 divider that maps the full charge back to the ESP32 ADC. A 0.1 μ F shunt capacitor implements a low-pass filter to limit voltage ripple to less than 100 mV p-p, yielding stable, reliable, repeatable conversions. The ADC on pin GPIO4 can parse out $2^{12} = 4096$ different values, or roughly 6.15 mV steps in the calculated battery voltage. In this, it can be reliably stated that the ADC can support 0.75 V in granularity for indicating the battery's state of charge. These calculations can be seen under the power_fc_asig table.

Actuation is provided via 5 PWM signals, 1 for each motor. These signals each operate at 400 Hz with 40-80% duty and 3.3V/0V logic levels that meet the ESC MCU (STM32) logic level. The varying duty cycle controls the motors indirectly through the STM32.

BOOT/RESET hardware is external to avoid false presses under vibration conditions commonly found in any flying aircraft. Furthermore, an external OTG flasher allows for firmware updates without adding mass to the aircraft. This can be seen in Figure 8.

Collectively, the regulated 3.3 V supply with sufficient power supply, low noise battery sensing, UART IBUS commands, and 400 Hz PWM output to form a clear control chain from input to rotating propellers. The defined verification procedures confirm the values found in the table below.

3.5.4.Interface Validation

Interface property *Be sure to use the naming convention "from_to_type" here.	Why is this interface property this value?	How do you know your design details will meet or exceed this property? Cite your sources in IEEE.
--	--	---

power_fc_dcpwr: Input

Vmin: 3.10V	3.0V is the minimum amount of voltage that the ESP32 Microcontroller, BMP581 Barometer and BNO085 IMU	The AP6320 buck converter's 3.3V output is above this minimum voltage range [14].
-------------	---	---

	can have input [1, p. 29][2, p.10][3, p.35]. A small safety margin of 0.1V is added to ensure proper operation.	
Vmax: 3.45V	This is the maximum amount of voltage that the ESP32 Microcontroller, BMP581 Barometer and BNO085 IMU can have input [1, p. 29][2, p.10][3, p.35]. A small safety margin of 0.15V is subtracted to ensure proper operation.	The AP6320 buck converter's 3.3V output is below this maximum voltage range [14].
Inominal: 50 mA	Normal operation draws a typical 355 mA, based on the active-mode specification where all ESP32 peripherals are enabled, including Bluetooth and Wi-Fi [1, p. 30]. The BMP581 and BNO085 add only 260 μ A and 100 μ A, which is negligible [2, p. 12][3, p. 51]. Bench testing shows a much lower draw of X mA, consistent with Bluetooth and Wi-Fi being disabled on the ESP.	The AP6320 buck converter's 2A output is above this nominal current range [14].
Ipeak: 100 mA	The ESP32's datasheet lists a maximum draw of 500 mA [1, p. 29]. Manual testing never exceeded 50 mA, and applying a x2 safety factor gives 100 mA.	The AP6320 buck converter's 2A output is above this nominal current range [14].

power_fc_asig: Input

Vrange: 0-25.2	This represents the voltage range output on the analog signal line. It depends on the charge level of the 6S LiPo battery, where each of the six	A voltage divider is utilized to linearly step down the asig signal by 2/17 (as seen in Figure x), reliably providing a safe voltage range for the
----------------	--	--

	cells can charge up to a maximum of 4.2 V [4]. $6 \text{ cells} \times 4.2 \text{ V} = 25.2 \text{ V}$	ESP32 <u>under</u> our operating voltage [1, p. 29]. $V_{out} = \frac{2}{17} V_{in}$ $V_{out}(25.2) \approx 2.96 \text{ V}$ $V_{out}(0) = 0 \text{ V}$
Minimum Sampling Frequency: ≥ 1 Hz	This is the minimum rate at which the ADC samples battery voltage. Providing sufficient resolution to capture slow, gradual battery discharge behavior. This low frequency minimizes processing overhead.	A 1 Hz sampling frequency is well within the capabilities of the ESP32 [1, p.28].
Frequency: 0 Hz	The battery (or DC supply) provides a DC voltage that does not oscillate, so its fundamental frequency is effectively 0 Hz.	The ESP32-S3 ADC supports DC and low-frequency analog inputs, so reading a 0 Hz signal is fully within its capabilities [1, p.28].
Read Resolution: 0.75 V	This represents the voltage resolution of the battery measurement. Such variation is negligible in terms of motor performance and battery life.	The ESP32 GPIO4 pin uses a 12-bit ADC, providing 2^{12} possible digital values. For an input range of 0V to 25.2 V, the corresponding voltage step size is: $(25.2 - 0) \div (2^{12} - 1)$ $\sim 6.15 \text{ mV}$ This means the ADC can distinguish changes as small as 6.15 mV. For readability and practical monitoring, the value will be rounded to increments of 0.75 V.

communication_fc_dsig: Input

Protocol: UART IBUS	The FS-iA6B uses the IBUS protocol implemented over	Confirmed in FlySky's official specifications, which state
------------------------	---	--

	UART, enabling high-speed digital communication and telemetry support. IBUS packets are serialized using UART [10].	IBUS uses UART electrical signaling at 115200 bits per second [11].
Datarate: 115200 baud	115200 is the bit rate supported by this transceiver [11]. A higher bit rate ensures low latency for user input.	Common bit rate supported by most microcontrollers including the ESP32 [1]
Logic Level: 3.1~3.45 V	3.3 V logic level compatible with most MCUs (ESP32, STM32).	The datasheet for the FS-iA6B states 3.3V logic level [10].
Packet width: 32 bytes (14 channels + checksum)	32 bytes is defined by the IBUS protocol specification to accommodate 14 channels of 16-bit data plus a checksum [12]. More than enough channels to control 5 motors.	Confirmed through FlySky documentation [12]

fc_prop_dsig: output

Logic High: 3.1~3.45 V	The flight controller (ESP32) operates at 3.3V [6, p.29], so its GPIO high level is equal to its supply voltage.	The ESC MCU (STM32F030F4P6) correctly interprets any $input > 0.7 \times VDD$ as high [6, p.61], where $VDD = 3.3V$. So $Logic\ high > 0.7 \times 3.3 = 2.31V$ is valid. The Flight control MCU logic high 3.3V is well above that.
Logic low: 0V	The flight controller (ESP32) drives 0V for logic low states.	The ESC MCU (STM32F030F4P6) correctly intercepts any $input > 0.3 \times VDD$ as low [1, p.61], where $VDD = 3.3V$. So $Logic\ low > 0.3 \times 3.3 = 1V$ is valid. The Flight control MCU logic low 0V is well below that.

PWM frequency: 400 Hz	This control frequency refers to how often the flight controller (ESP32) sends throttle commands to the ESC. In PWM based controlled systems, this frequency is typically between 50 and 490 Hz in modern implementations [7]. 400 Hz is a common PWM frequency to use.	The PWM control frequency will be generated by configuring the ESP32 timer. $f_{PWM} = f_{LED_CLK} / (LEDC_CLK_DIV \times 2^{LEDC_TIMERxDUTY_RES})$ [8, p.1313]. It is determined by the 80MHz peripheral clock using configurable settings. This allows PWM frequencies from below 1Hz up to MHz, well beyond the 50-490 Hz range required for the ESC control signals.
Duty Cycle: 40 to 80%	The duty cycle defines the percentage of time the PWM signal remains high within one period. For throttle control this value determines how much power is delivered to the motor. A full range from 40% (motor off) to 80%(full throttle) is chosen to provide a linear relationship between the PWM signal and the motor speed command.	The ESP32 microcontroller supports hardware PWM generation through the LEDC peripherals [1, p.25], allowing precise duty cycle control from 0 to 100%.

3.5.5.Verification Process

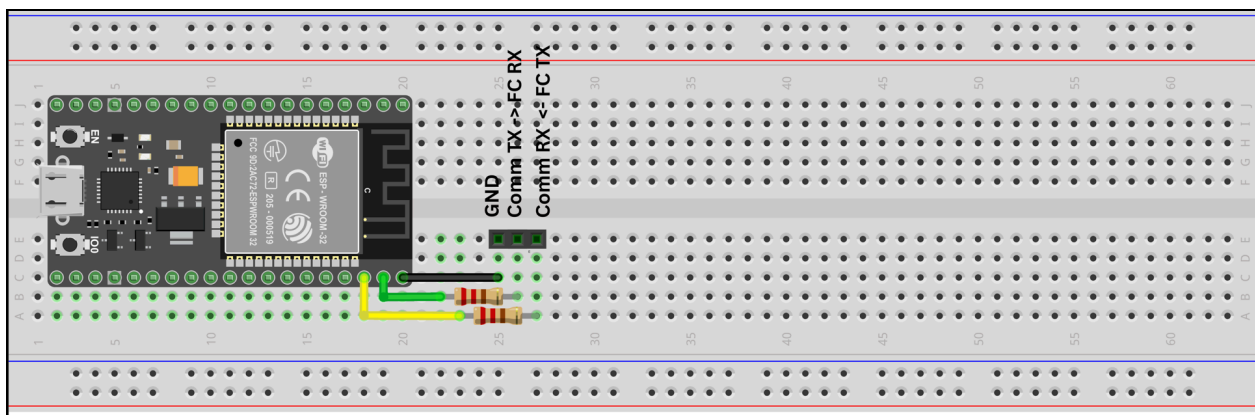


Fig. 15: Communication block emulation circuit configuration. Image created with Fritzing software [15]. The resistor color bands are not representative of the actual values. Both resistors should be 4.7 k Ω .

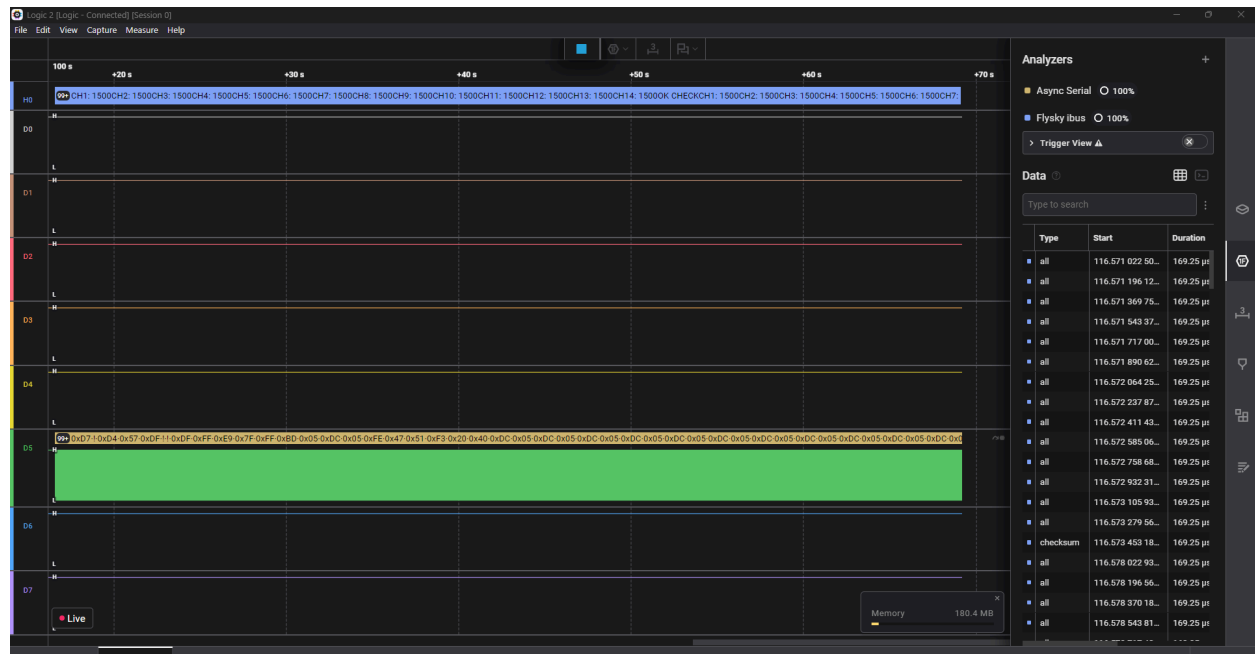


Fig. 16: Logic analysis of Flysky iBus UART packets.

```

♦♦♦ RC DATA ♦♦♦
[RC] valid=YES roll=0 pitch=0 yaw=0 thrRaw=0
+++ END FRAME +++

```

```

♦♦♦ RC DATA ♦♦♦
[RC] valid=YES roll=1500 pitch=1500 yaw=1500 thrRaw=1500
+++ END FRAME +++

```

Fig. 17: Before (left) and after (right) serial monitor output when turning on communication emulator.

Fig. 18: Async configuration for logic analyzer.

Tools Required:

- THC 4.7 k Ω resistor x2, ESP32 Development board, Breadboard, Micro-B USB cable.
- Multimeter, Oscilloscope, Logic Analyzer.
- Flight Controller PCB, Flasher OTG PCB, USB-C cable and cables in Fig. 14 & 15.

Setup:

1. Assemble the Communication block emulation circuit in Figure 15.
2. Flashing:
 - Flash ESP32 development board with test script found here: https://github.com/reefstarnes/Solar_Glide_VTOL/tree/main/Flightcontrol/testcode/comm_simulate
Note: Power the ESP32 down after flashing.
 - Flash the Flight Controller (FC) utilizing the π cable with test script found here: https://github.com/reefstarnes/Solar_Glide_VTOL/tree/main/Flightcontrol/testcode/flight_control_test
Note: The FC is most likely already flashed
3. Utilizing Cable α , connect the FC to the circuit built in step #1.
4. Utilizing Cable β , connect the FC to the multi-channel bench power supply, routing Red $\rightarrow$ CH1+, White $\rightarrow$ CH2+, and Black $\rightarrow$ CH1- & CH2-. This will act as the Power block for the drone. For now set both channels to 0V and set them off.
5. Utilizing Cable Δ , connect the FC to the IMU breakout board (Fig. 12).

Verify power_fc_dcpwr

1. For each test, keep power flowing for greater than 15 seconds to ensure stable voltage and current readings.

2. Set CH1 on the power supply to 3.1 V ($V_{\min}$) and turn on. Confirm the following holds true no short of 15 seconds:
 - Confirm that the ESP32 is functioning via the red flashing indicator LED and the voltage does not drop below 3.1 V.
 - Confirm that the current is near 50 mA (I_{nominal}) and below 100 mA (I_{peak}) .
3. Set CH1 on the power supply to 3.45 V ($V_{\max}$) and turn on. Confirm the following holds true no short of 15 seconds:
 - Confirm that the ESP32 is functioning via the red flashing indicator LED and the voltage does not drop below 3.45V.
 - Confirm that the current is near 50 mA (I_{nominal}) and below 100 mA (I_{peak}).

Verify fc_prop_dsig

1. Connect the FC to Prop Controller via Cable E.
2. Power on the FC by setting CH1 to 3.3 V (or any voltage within 3.1 and 3.45 V) and turn on.
3. Verify logic levels:
 - Join the oscilloscope probe to oscilloscope and turn on.
 - Use probes on any one of the 5 test pads adjacent to the JST GH 6 pin connector. This may require soldering a jumper to the test pad for oscilloscope probes to properly attach.
 - Confirm that the logical high level is approximately 3.3V and the logical low is 0V via cursor A and cursor B on the oscilloscope.
4. Verify duty cycle and frequency:
 - Connect the logic analyzer to your PC with USB.
 - Attach the female side of a male-to-female jumper cable to any logic-analyzer channel, and connect ground with a female-to-female cable between the analyzer and the PCB.
 - Start the capture in the logic-analyzer software, then probe each test pad next to the six-pin connector for at least one second using the male end of the jumper cable.
 - In the software, confirm the frequency is 400 Hz. One pad should show D = 40, and another should show D = 80.

Verify communication_fc_dsig

1. Power on the FC by setting CH1 to 3.3 V (or any voltage within 3.1 and 3.45 V) and turn on.
2. Utilizing Cable λ , connect the FC to the Flasher OTG to your PC via USB-C.
3. Open the serial monitor for the FC, and do the following simultaneously:
 - Plug in the circuit made in setup #1 via USB-B.
 - Check the serial monitor output changes as specified in Fig. 17.

4. Probe the Comm TX pin, as seen in Fig 15, with an oscilloscope probe to determine logic levels of 0V for low and 3.3V for high.
5. Use the logic analyzer on channel 5 to confirm the packet length of 32 bits, BAUD 115200 and UART iBUS is properly, sending each channel at 1500, as seen in Fig. 16. This is done by setting Async serial to the values specified in Fig 18.

Verify power_fc_asig

1. Utilizing Cable λ, connect the Flight Controller to the Flasher OTG to your PC simply for serial motoring.
2. Set multimeter to probe battery input voltage (white cable).
3. Power on the Flight Controller by setting CH1 to 3.3 V (or any voltage within 3.1 and 3.45 V), CH2 to 0V and turn on.
4. Check multimeter voltage reading is stagnant at a single voltage, indicating 0 Hz frequency.
5. Verify that the ADC readings in the serial monitor can consistently update at or greater than 1Hz.
6. Repeat steps 3-5 for CH2+ = 5, 15, 22.5, 25.2 Volts.

3.5.6.References

- [1] Espressif Systems, "ESP32-S3-WROOM-1 & WROOM-1U Datasheet (v1.5)", Doc. No. ESP32-S3-WROOM-1_WROOM-1U_Datasheet_EN, 4.0 years (≈2021) old. [Online]. Available:
https://www.espressif.com/sites/default/files/documentation/esp32-s3-wroom-1_wroom-1u_datasheet_en.pdf
- [2] Bosch Sensortec GmbH, "BST-BMP581-DS004-13 Barometric Pressure Sensor – Datasheet", doc. no. BST-BMP581-DS004-13, Apr. 2025. [Online]. Available:
<https://www.bosch-sensortec.com/media/boschsensortec/downloads/datasheets/bst-bmp581-ds004.pdf>
- [3] CEVA Inc., "BNO080/085 Datasheet – 9-Axis SiP Sensor Hub Family", Doc. No. 1000-3927 v1.17, 2023. [Online]. Available:
https://www.ceva-ip.com/wp-content/uploads/BNO080_085-Datasheet.pdf
- [4] ChinaHobbyLine, "CNHL Racing Series 6200 mAh 22.2 V 6S 90 C LiPo Battery with EC5 Plug," ChinaHobbyLine [Online]. Available:
<https://chinahobbyline.com/collections/cnhl-voltage-22-2v-6s-lipo-batteries/products/cnhl-racing-series-6200mah-22-2v-6s-90c-lipo-battery-with-ec5-plug>
- [5] Espressif Systems, ESP32-S3 Hardware Design Guidelines, Release master, Sep. 25, 2025. [Online]. Available:

- <https://docs.espressif.com/projects/esp-hardware-design-guidelines/en/latest/esp32s3/esp-hardware-design-guidelines-en-master-esp32s3.pdf>
- [6] STMicroelectronics, “STM32F030x4/STM32F030x6/STM32F030x8/STM32F030xC Value-line ARM®-based 32-bit MCU with up to 256 KB Flash, timers, ADC, communication interfaces, 2.4-3.6 V operation, Datasheet DS00088500 Rev 5,” STMicroelectronics, Geneva, Switzerland, Nov. 2021. [Online]. Available: <https://www.st.com/content/ccc/resource/technical/document/datasheet/a4/5d/0b/0e/87/c4/4d/71/DM00088500.pdf/files/DM00088500.pdf/jcr:content/translations/en.DM00088500.pdf>. Accessed: Nov. 2, 2025.
- [7] G. Staff, “What is an Electronic Speed Controller & How does an ESC work,” Tyto Robotics. [Online]. Available: <https://www.tytorobotics.com/blogs/articles/what-is-an-esc-how-does-an-esc-work>. Accessed: Nov. 2, 2025.
- [8] Espressif Systems, “ESP32-S3 Technical Reference Manual,” [Online]. Available: https://www.espressif.com/sites/default/files/documentation/esp32-s3_technical_reference_manual_en.pdf. Accessed: Nov. 3, 2025.
- [9] Espressif Systems (Shanghai) Co., Ltd., “ESP32-S3-DevKitC-1 v1.1 User Guide,” ESP-Dev-Kits, Nov. 2025. [Online]. Available: https://docs.espressif.com/projects/esp-dev-kits/en/latest/esp32s3/esp32-s3-devkitc-1/user_guide_v1.1.html#getting-started
- [10] Bill M. Preston, “Radio Control – Use the Flysky FS-I6X with Arduino & Build an RC Car,” DroneBot Workshop, 2021. [Online]. Available: <https://dronebotworkshop.com/radio-control-arduino-car/>
- [11] FlySky, “FS-iA6B Receiver Specifications,” FlySky Model Co., Ltd., 2023. [Online]. Available: <https://www.flysky-cn.com/ia6b-canshu>
- [12] Will. “IBUS and Arduino.” *The Nerdy Engineer*, 21 Nov. 2023. [Online]. Available: <https://thenerdyengineer.com/ibus-and-arduino/>
- [13] Texas Instruments, “TPS56A37: 4.5-V to 36-V, 6-A Synchronous Step-Down Converter (buck),” datasheet (specification), [Online]. Available: <https://www.ti.com/lit/ds/symlink/tps56a37.pdf>
- [14] AP63200/AP63201/AP63203/AP63205 document number: DS41326 rev. 3 - 2 1 of 18, <https://www.diodes.com/assets/Datasheets/AP63200-AP63201-AP63203-AP63205.pdf> (accessed Nov. 3, 2025).

[15] Fritzing Team, Fritzing: Open-Source Electronics Design and Prototyping Tool. Accessed: Feb. 28, 2025. [Online]. Available: <https://fritzing.org>

[16] C. Nelson, "Working with I2C devices," Adafruit Learning System, <https://learn.adafruit.com/working-with-i2c-devices/pull-up-resistors> (accessed Dec. 3, 2025).

3.6. Aircraft-Body

Block owner is Reef Starnes

The Aircraft-body block defines the physical geometry and aerodynamic surfaces required for lift, stability, and control during both VTOL and forward flight modes. It includes the wing, fuselage, and twin-boom tail. There are many geometric parameters to take into consideration like wingspan, wing area, aspect ratio and weight. The approach for design was using simplified mathematical models for design and cross checking with Autodesk CFD.

3.6.1. Description

The aircraft body adopts a fixed-wing, high-wing configuration with a pusher propeller mounted on the rear of the fuselage. A high-wing means that the wing is mounted towards the top of the fuselage, and it was chosen in this case because it offers more stability than mid or low wing configurations [5].

The tail of the craft utilizes a twin-boom tail, which carries both horizontal and vertical stabilizers. We selected a twin-boom tail because this layout is commonly paired with a pusher propeller; without the separated booms, the fuselage would obstruct the propeller wake. Furthermore, our design already employs two structural tubes for the quad motors, so using them as tail booms creates a dual-purpose component that can reduce added drag and avoid the extra structure that would be needed to mount say a T-tail. All tails use symmetrical foils due to their predictable, symmetrical lift [9][8]. A NACA 0010 is a commonly used foil for this purpose and what we will use [9][8]. To design for an effective horizontal stabilizer we use Equation #3, where based on the 1 meter long boom, at a wing mounted in the center, we can confidential say the horizontal stabilizer will be about 0.5 meters from the center of mass. Along with a reasonable coefficient of 0.3:

$$V_h = \frac{S_h l_h}{S_w c_{w,MAC}} \rightarrow \frac{S_h^{0.5}}{(0.318 m)(0.25 m)} = 0.3 \rightarrow S_h = 0.0477 m^2 \text{ (a.k.a. horizontal stabilizer area)}$$

Then selecting a span of 0.5 meters, results in 95.4 mm chord length. The vertical stabilizer had the same approach using Equation #2, $V_h = 0.03$.

The quad motors are mounted to the carbon spar using 4 3D printed u-clamps, for easy adjustability when aligning the quad motors with the center of mass. The fuselage designed around the layout of electronics needed to be housed within it, and was designed last. It is designed for the center of mass aligned with the wings ~25% chord line, a common design choice which can be seen in Figure 9 [10]. Scratchwork and further reasoning for design choices can be found in the notes document: [Aircraft_body_notes_V2](#)

3.6.2.Design

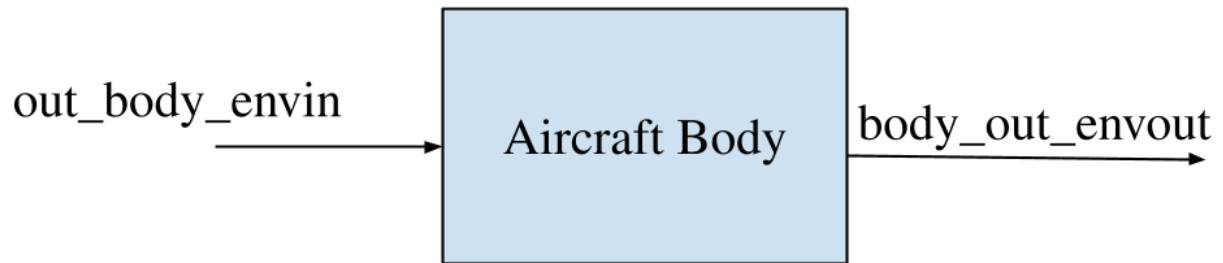


Fig. 1: Black box diagram of the flight controller and its interfaces.

$$\frac{W}{S} = 0.5 \rho V_{stall}^2 C_{L,max} \rightarrow S = [0.5 \rho V_{stall}^2 C_{L,max}]^{-1} W$$

Equation 1: Stall Wing-Loading Sizing Relation. This formula connects stall speed, maximum lift coefficient, and air density to the allowable wing W/S , and is used to size the required wing area for the aircraft [4, p48]. Where W is weight, S is wing area, ρ is air density (1.225 kg/m³), V_{stall} is stall speed, and $C_{L,max}$ is maximum lift coefficient.

$$\frac{b^2}{S} = AR$$

Equation 2: Aspect ratio equation given wing area S and span b .

$$V_v = \frac{S_{v,total} l_v}{S_w b_w} = \frac{2S_{v,single} l_v}{S_w b_w}$$

Equation 3: Horizontal tail volume coefficient formula. A formula for essentially how far l_v (tail moment arm) and how big S_h (area) the horizontal tail wing should be to be effective. Should be within 0.02 and 0.05 for a well behaved aircraft [12, p.4].

$$V_h = \frac{S_h l_h}{S_w b_w}$$

Equation 4: Similar to equation 3, but an effective stable range is 0.3 to 0.6 for this horizontal volume coefficient [12, p.4].

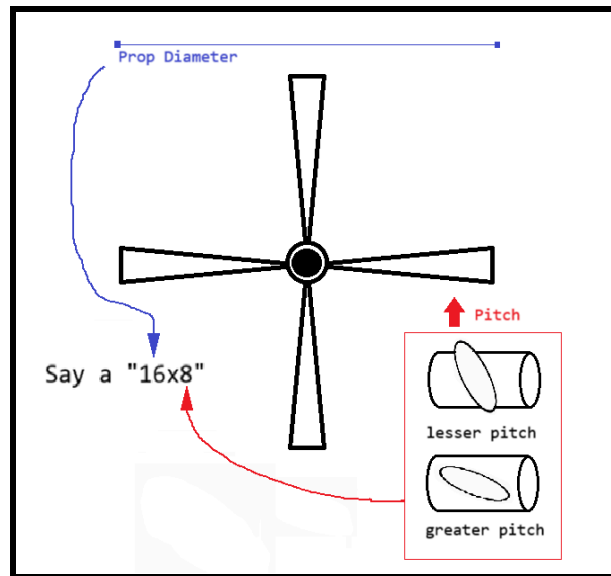


Figure 2: How prop sizes work.

Component	Qty	Weight contribution	Other Notes
Battery	1	902g $\pm$ 5g -> 902g	
Q_Motor	4	$52.36 * 4 = 209.44$ g	
P_Motor	1	115 g	
Servo	4		
ESC_PCB	4	$50 * 4 = 200$	
FC_PCB	1	30	
COM_PCB	1	25	
PWR_PCB	1	70	
Carbon_Spar	2	$2 * 125 = 250$ g	
Tot.	1	1,801.44	4312 - x = z short of max weight

Figure 3: Weight estimation for the entire craft. Due to the thrust capabilities of the drone and a 1.5 thrust-to-weight ratio we must remain below about 4.3 kg net weight, allowing us to calculate wingspan using Equation 1 for the worst case scenario.

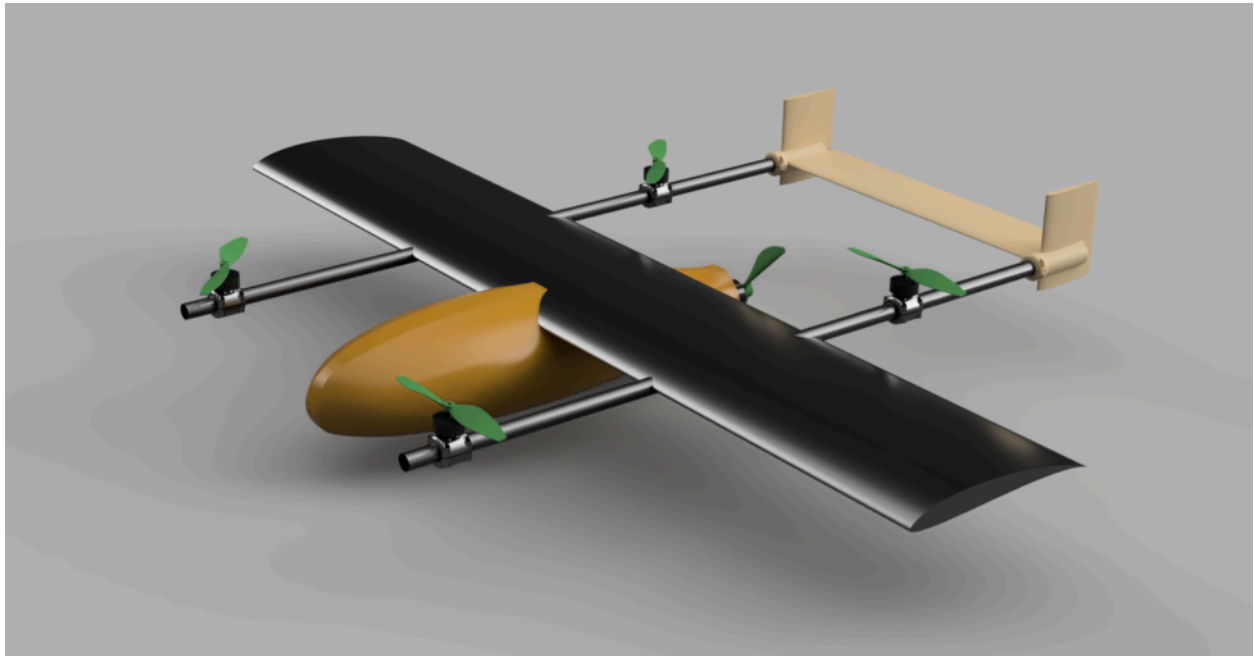


Figure 4: Autodesk Fusion render of aircraft.

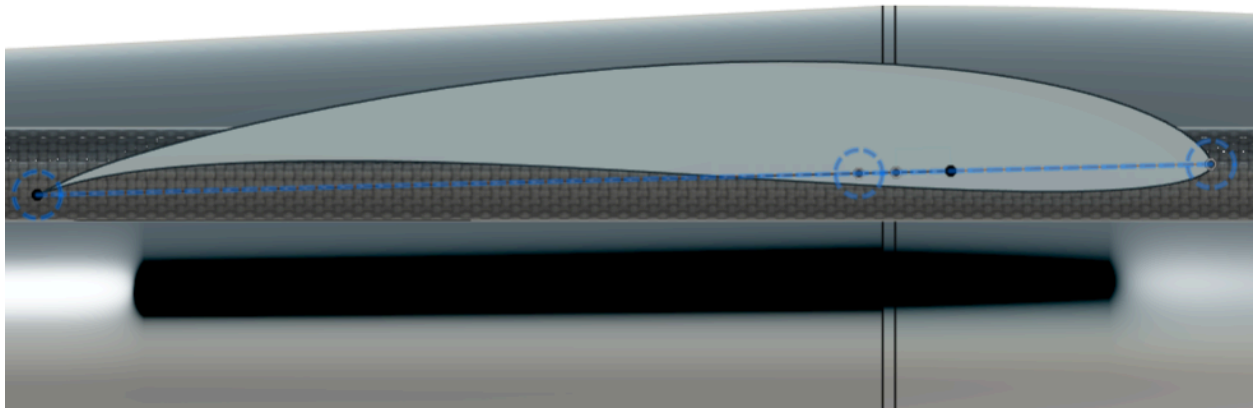


Figure 5: Closeup view of E216 airfoil with 1.5 degrees angle of incidence. This foil was selected as it had the greatest max(Cl/Cd) 8% - 14% thickness on the airfoil database I used, Air Foil Tools [6]. Note this thickness range was chosen for wing strength. The 1.5 angle of incidence was chosen to reduce pitch when cruising and shorter take off times, which appears to be a common value utilized but CFD will tell us more [7]. The circle sketched in the middle is the ~25% chord mark.



Figure 6: Custom motor models based on the motor data sheets for the quad motor (left) [2] and pusher motor (right) [1]. The primary use is to verify clearances, drag contribution in when simulating with CFD and creating the mounting holes and brackets.

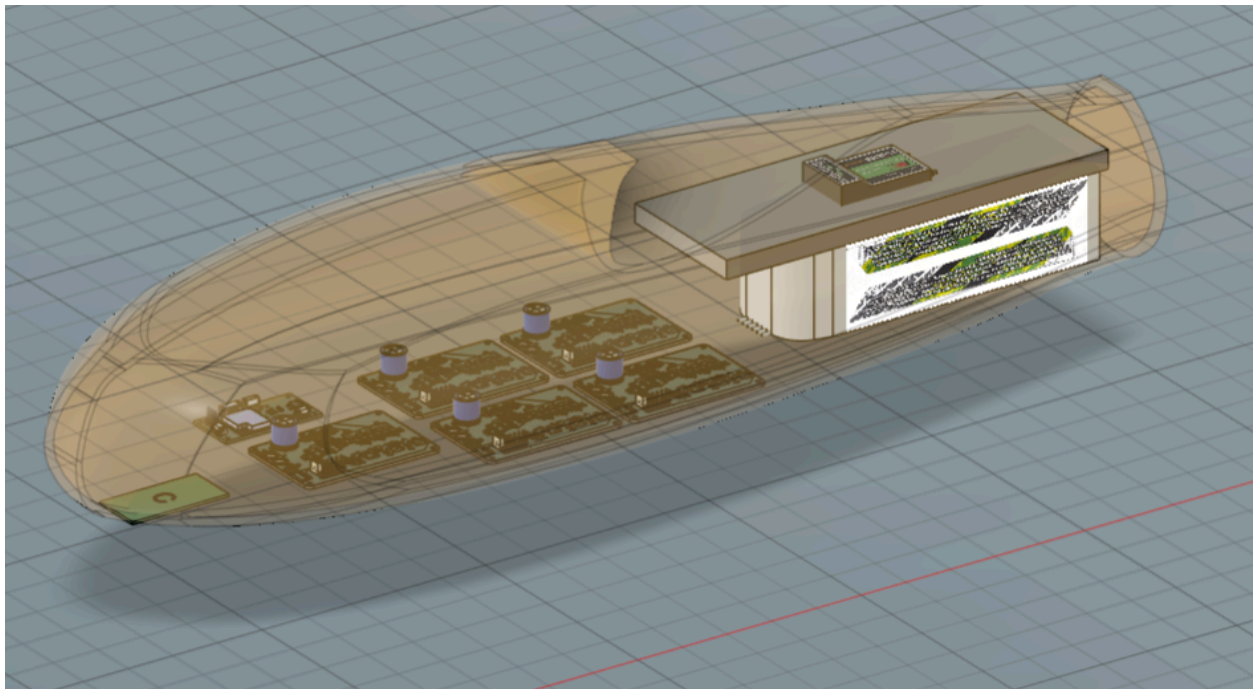


Figure 7: Imported PCBs from Kicad and custom battery model in preparation for layout configuration for fuselage design (last step). Note what is missing here which will take up real estate is the transceiver (in-progress) and power distribution board (in progress).

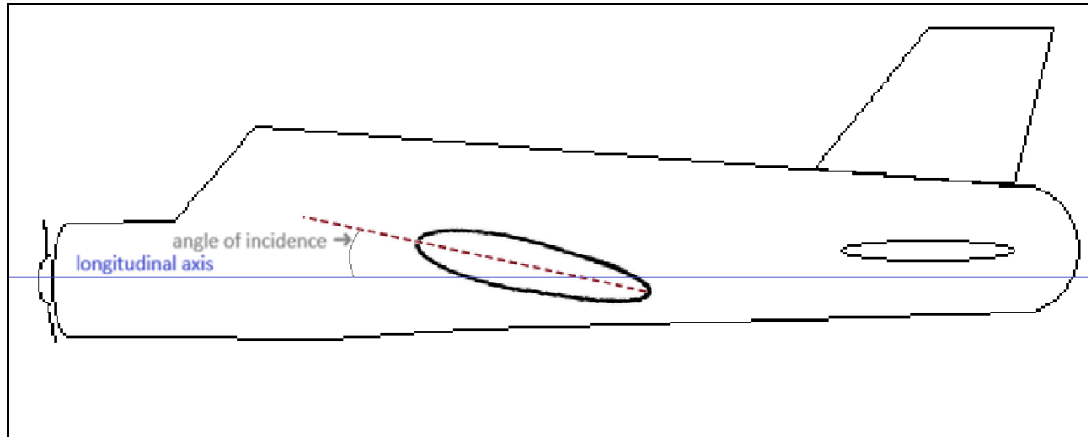


Figure 8: Angle of incidence visualized.

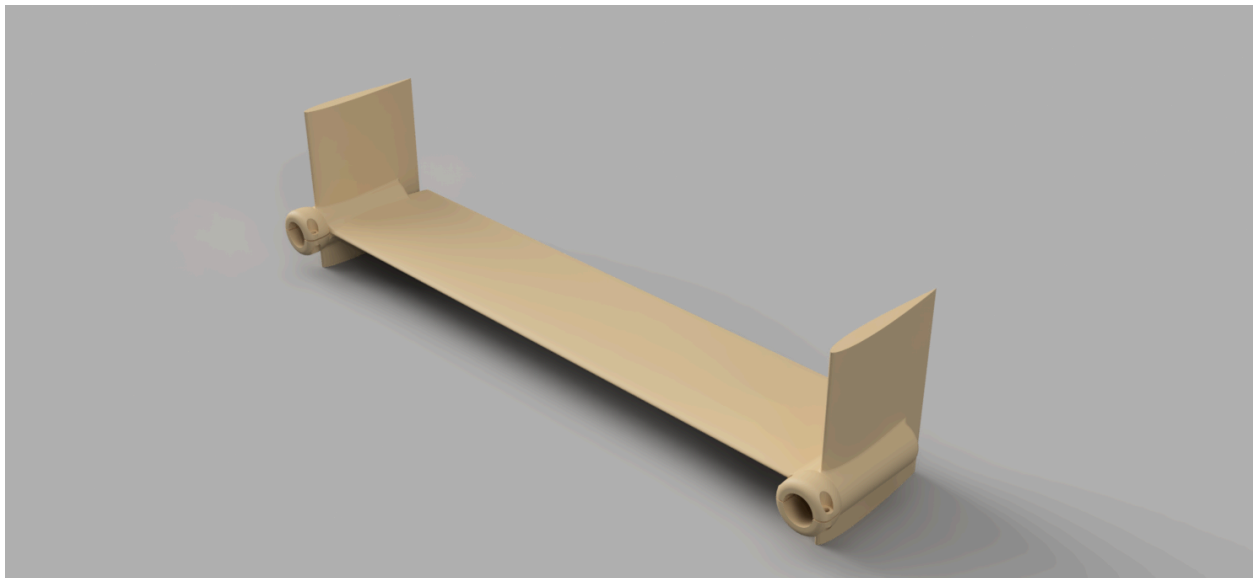


Figure 9: Autodesk fusion tail rendering.

3.6.3. General Validation

The wing area was first calculated using equation 1, using our calculated stall speed of 12.9 m/s (with $C_{L,max} = 1.3$). However after picking the foil we found our $C_{L,max} = 1.6$ [6] , essentially overbuilding the wing to generate lift at an even lower speed of 11.3 m/s when solving for V_{stall} with the same configuration. The relationship in this equation tells us a lot, that the further we reduce weight, we reduce the stall velocity by the same factor. The AR of ~4.9 was chosen as anything greater increased the span beyond 1.25 M, already quite large, and calculated via equation 2. The Nose-Tail length was chosen because of sizing of carbon fiber rods, which often come in either 0.5 M or 1 M, the former was more feasible.

3.6.4.Interface Validation

Interface property *Be sure to use the naming convention “from_to_type” here.	Why is this interface property this value?	How do you know your design details will meet or exceed this property? Cite your sources in IEEE.
--	--	---

out_body_envin: Input

Motor _Q -Mount Bolt: [M3 * 8] x 16	According to its datasheet [1], this brushless motor requires four M3 bolts for mounting.	Metric bolt sizes are a reliable standard. 3D prints generally offer ± 0.1 mm accuracy [3] which is sufficient for our design.
Motor _Q -Mount Mounting holes: [M3.1 $\pm$ 0.1] x 16	According to its datasheet [1], this brushless motor requires four M3 bolts for mounting.	The design of the associated mounting holes (3.1 mm) are large enough for an M3 bolt.
Mount-Boom Bolt: [M4 * 16] x 16	A larger size of M4 over M3 was chosen to more reliably clamp onto the boom.	Metric bolt sizes are a reliable standard. 3D prints generally offer ± 0.1 mm accuracy [3] which is sufficient for our design.
Mount-Boom Mounting holes: [M4.1 $\pm$ 0.1] x 16	Required holes for M4 Mount-Boom bolts.	The design of the associated mounting holes (4.1 mm) are large enough for an M4 bolt.

body_out_envout

Horizontal stabilizer chord length: 95.4 $\pm$ 3.5 mm	Based on the calculations in the design section.	Calculated in the design section to have a stable V_h of 0.3 [12, p.4].
Horizontal wing spar length: 0.4 m.	Approximately the span of the horizontal stabilizer, as calculated in the design section.	Calculated in the design section to have a stable V_h of 0.3 [12, p.4].
Boom length: 1 $\pm$ 0.10 m	The convenience and cost effectiveness of pre-made carbon fiber tubes on the market come in	Design details will meet because of data sheet for carbon fiber tube [11]

	this typical standard size. This forms the horizontal frame of the aircraft.	
--	--	--

3.6.5.Verification Process

Setup:

1. Acquire and turn on calipers for measuring.
2. Acquire tape measure

Testing:

1. Measure the following holes and assure they are within tolerance using calipers:
 - A. Motor_Q-Mount mounting holes: 3.1 ± 0.1 mm (Qty: 16)
 - B. Mount-Boom mounting holes: 4.1 ± 0.1 mm (Qty: 4)
 - C. Mount-Boom Bolt: [M4 * 16] x 16
 - D. Motor_Q-Mount Bolt: [M3 * 8] x 16
2. Measure the following lengths using a tape measure.
 - A. Horizontal stabilizer chord length: 95.4 ± 3.5 mm. This is the distance between the leading edge and trailing edge of the foil.
 - B. Horizontal wing spar length: 0.4 m.
 - C. Boom length: 1 ± 0.05 m

3.6.6.References

[1] GEPRC EM3115 3115 900KV 6S Brushless Motor for 8–9–10 Inch Long Range FPV RC Racing Drone, Banggood, 2026. [Online]. Available: <https://www.banggood.com/GEPRC-EM3115-3115-900KV-6S-Brushless-Motor-for-8-9-10-Inch-Long-Range-FPV-RC-Racing-Drone-p-2003873.html> . Accessed: Jan. 22, 2026.

[2] GEPRC EM3115 3115 900KV 6S Brushless Motor for 8–9–10 Inch Long Range FPV RC Racing Drone, Banggood, 2026. [Online]. Available: <https://www.banggood.com/GEPRC-EM3115-3115-900KV-6S-Brushless-Motor-for-8-9-10-Inch-Long-Range-FPV-RC-Racing-Drone-p-2003873.html> . Accessed: Jan. 22, 2026.

[3] Formlabs, “Guide to 3D Printing Tolerances, Accuracy, and Precision,” Formlabs Blog. [Online]. Available: <https://formlabs.com/blog/understanding-accuracy-precision-tolerance-in-3d-printing/>. [Accessed: January 22, 2026].

[4] T. R. Yechout, S. L. Morris, D. E. Bossert, and W. F. Hallgren, *Introduction to Aircraft Flight Mechanics: Performance, Static Stability, Dynamic Stability, and Classical Feedback Control*,

American Institute of Aeronautics and Astronautics, Inc., 2003. [Online]. Available: <https://avioniconorwegian.de/documents/AerospaceDocs/IntroAcftFlightMech.pdf>. [Accessed: Jan. 22, 2026].

[5] N. Wolfe, "What is the difference between high wing, mid wing, and low wing?," Ask an Explainer, Smithsonian National Air and Space Museum, Nov. 1, 2012. [Online]. Available: <https://howthingsfly.si.edu/ask-an-explainer/what-difference-between-high-wing-mid-wing-and-low-wing>. [Accessed: Jan. 22, 2026].

[6] "E216-IL Airfoil Details," Airfoil Tools. [Online]. Available: <http://www.airfoiltools.com/airfoil/details?airfoil=e216-il>. [Accessed: Jan. 22, 2026].

[7] "What is the reason of having an Angle of Incidence on an airplane?," Aviation Stack Exchange, Oct. 21, 2016. [Online]. Available: <https://aviation.stackexchange.com/questions/32573/what-is-the-reason-of-having-an-angle-of-incidence-on-an-airplane>. [Accessed: Jan. 22, 2026].

[8] McDonald, R., "What type of airfoil should be used for horizontal tailplanes?," Aviation Stack Exchange, Jul. 8, 2024. [Online]. Available: <https://aviation.stackexchange.com/questions/105868/what-type-of-airfoil-should-be-used-for-horizontal-tailplanes>

[9] A. Wood, "Aircraft Horizontal and Vertical Tail Design," AeroToolbox, Sep. 28, 2022. Online. Available: <https://aerotoolbox.com/design-aircraft-tail/>

[10] Wikipedia, "Aerodynamic center," Wikipedia, The Free Encyclopedia, Aug. 10, 2025. [Online]. Available: https://en.wikipedia.org/wiki/Aerodynamic_center. Accessed: Jan. 22, 2026. (defining the point on an airfoil where the pitching moment coefficient is invariant (unchanging) with respect to angle of attack).

[11] SpeedyFPV, "1 Meter Carbon Fiber Tube Roll Wrapped," SpeedyFPV, Accessed (retrieved): Feb. 13, 2026. [Online]. Available: <https://speedyfpv.com/products/1000mm-3k-carbon-fiber-tubes-16-18-20-25mm-matte-glossy-1mm-2mm-walls?variant=45508748247254>

[12] M. Drela et al., "Lab 8 Notes – Basic Aircraft Design Rules," Massachusetts Institute of Technology Unified Engineering I, II, III & IV, Fall 2005–Spring 2006. [Online]. Available: https://ocw.mit.edu/courses/16-01-unified-engineering-i-ii-iii-iv-fall-2005-spring-2006/ffbff8a4985ee6ea819afb90c40834e3_spl8.pdf. Accessed: Jan. 22, 2026.

4. Project Closing

This section will begin with an overview paragraph(s) that summarizes the contents of this section. This paragraph must reference the satisfaction of requirements (or not), the current state of the project, and any challenges that were encountered in implementing the project and how they were addressed (this might be how they were resolved, removed, or accepted by the team and project partner).

4.1.Future Recommendations

4.1.1.Technical recommendations

This section needs to contain **at least 4 technical recommendations** for a future team continuing this project. Explain the concern fully and include specific suggestions a following team could start with to solve the problem. Be sure to include in-text citations.

4.1.2.Global impact recommendations

This section needs to contain **at least 2 global impact recommendations** for a future team continuing this project. Explain the concern fully and include specific suggestions a following team could start with to solve the problem. Be sure to include in-text citations.

4.1.3.Teamwork recommendations

This section needs to contain **at least 2 teamwork/communication recommendations** for a future team continuing this project. Explain the concern fully and include specific suggestions a following team could start with to solve the problem. Be sure to include in-text citations.

4.2. Project Artifact Summary with Links

A comprehensive summary of all project artifacts with descriptive names and hyperlinks to accessible document files. This should be organized in a way the reader can easily follow. This section is your opportunity to demonstrate deep learning beyond what you put in the actual block details section.

4.3. Presentation Materials

A comprehensive summary of all presentation materials, organized in a way the reader can easily follow, with descriptive names and hyperlinks to accessible document files as well as an image of the team's project poster directly embedded in the section.

Noah Amack

Carlos D Alvarado-Lopez

Reef Starnes

4.4. References

This section should contain links to external references in **IEEE format**. These references only pertain to the contents of section 4, and no other sections.

A. Appendix

Any additional materials you would like to include. This is a placeholder, an appendix is not required.